

VIDYA JYOTHI
INSTITUTE OF TECHNOLOGY
AN AUTONOMOUS INSTITUTION

[Accredited by NAAC & NBA, Approved by AICTE New Delhi & Permanently Affiliated to JNTUH]

Aziz nagar Gate, Chilkur Balaji Road, Hyderabad - 500075

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

DJANGO

Regulation: R25

II B.TECH- I SEMESTER

LAB MANUAL

DJANGO LAB

L	T	P	C
0	0	2	1

B.Tech II Year I Semester

Course Code: A253585

Course Outcomes:

At the end of the course, student will be able to:

1. Design and Develop a responsive Web Application using HTML & CSS.
2. Build a responsive Web Application using Java Script, CSS, Flex & Grid.
3. Demonstrate understanding of the Django Framework
4. Implement Mailing, Cache and Session Management in Django Framework
5. Analyze, Design, Develop and Implement complete Web Applications using Django Framework

LIST OF EXPERIMENTS:

1. Build a Responsive Web Application for Registration Form, which contains User Name, Password, Date of Birth, Gender, Mail-id, Contact Number, Address and Submit Button.
2. Write a JavaScript Program to Validate Registration Page using Regular Expression.
3. Build a Responsive Web Application for Shopping Cart with Registration, Login, Catalog and Cart Pages using CSS3 features, Flex and Grid.
4. Basics of Django Framework & Installation of required Software's.
5. Create a shopping cart web application and add a customized admin page for product catalog management for Experiment 3
6. Demonstrate Cache, Session Management for Student Login Application.
7. Create a Quiz Application using Django Framework along with SQLite3.

8. Create a Django application using generic views and forms
9. Create an Application for Working with Mails using Django Framework
10. Create an application that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js implement using Django
11. **Case Study:** Django Application for VJIT Student Portal and deploy it into GitHub.

REFERENCE BOOKS:

1. "Web Technologies: HTML, CSS, JavaScript" *Uttam K. Roy*
2. "Django for Beginners: Build Websites with Python and Django" *William S. Vincent*
3. "Python Web Development with Django" *Jeff Forcier, Paul Bissex, Wesley Chun*

Web URLs:

1. W3Schools Web Development Tutorials
   https://www.w3schools.com
2. JavaScript Regular Expressions Guide – MDN
   https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions
3. Python Official Documentation
   https://www.python.org/doc/
4. Python Tutorial (W3Schools)
   https://www.w3schools.com/python/
5. Django Official Documentation
   https://docs.djangoproject.com
6. Django Tutorials
   https://djangoproject.com/start/

INDEX

S.NO	Experiment	Page No.
1	**WEEK1:** Build a Responsive Web Application for Registration Form, which contains User Name, Password, Date of Birth, Gender, Mail-id, Contact Number, Address and Submit Button	4-12
2	**WEEK2:** Write a JavaScript Program to Validate Registration Page using Regular Expression.	13-16
3	**WEEK3:** Build a Responsive Web Application for Shopping Cart with Registration, Login, Catalog and Cart Pages using CSS3 features, Flex and Grid.	17-31
4	**WEEK4:** Basics of Django Framework & Installation of required Software's	32-38
5	**WEEK5:** Create a shopping cart web application and add a customized admin page for product catalog management for Experiment 3	39-49
6	**WEEK6:** Demonstrate Cache, Session Management for Student Login Application	50-56
7	**WEEK7:** Create a Quiz Application using Django Framework along with SQLite3	57-63
8	**WEEK8:** Create a Django application using generic views and forms	64-72
9	**WEEK9:** Create an Application for Working with Mails using Django Framework	73-91
10	**WEEK10:** Create an application that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js implement using Django	92-110
11	**WEEK11:Case Study:** Django Application for VJIT Student Portal and deploy it into GitHub	111-115

Week1: Build a Responsive Web Application for Registration Form, which contains User Name, Password, Date of Birth, Gender, Mail-id, Contact Number, Address and Submit Button.

PROGRAM:

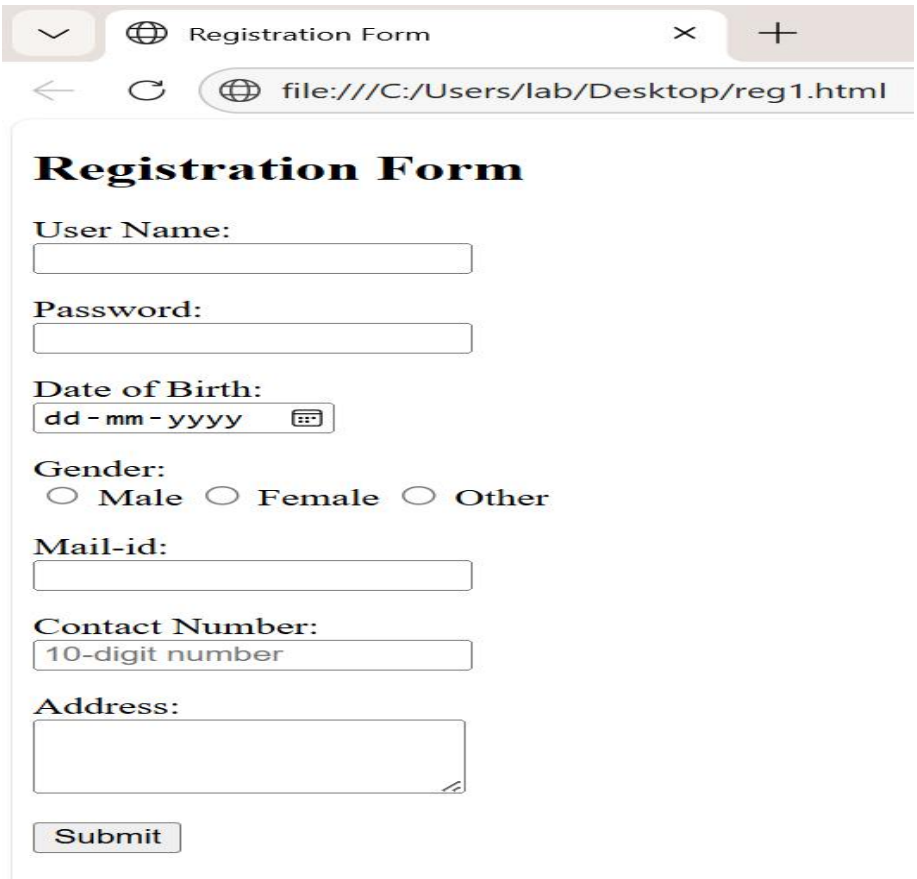
STATIC REGISTRATION FORM –HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form</title>
</head>
<body>
  <h2>Registration Form</h2>
  <form>
    <p>
      <label for="username">User Name:</label><br>
      <input type="text" id="username" name="username" required>
    </p>
    <p>
      <label for="password">Password:</label><br>
      <input type="password" id="password" name="password" required>
    </p>
    <p>
      <label for="dob">Date of Birth:</label><br>
```

```
    <input type="date" id="dob" name="dob" required>
  </p>
  <p>
    <label>Gender:</label><br>
    <input type="radio" name="gender" value="male" required> Male
    <input type="radio" name="gender" value="female" required> Female
    <input type="radio" name="gender" value="other" required> Other
  </p>
  <p>
    <label for="email">Mail-id:</label><br>
    <input type="email" id="email" name="email" required>
  </p>
  <p>
    <label for="contact">Contact Number:</label><br>
    <input type="tel" id="contact" name="contact" pattern="[0-9]{10}" placeholder="10-digit number" required>
  </p>
  <p>
    <label for="address">Address:</label><br>
    <textarea id="address" name="address" rows="3" required></textarea>
  </p>
  <p>
    <button type="submit">Submit</button>
  </p>
```

```
  </form>
</body>
</html>
```

OUTPUT:



PROGRAM:

Responsive Web Application using CSS,JAVASCRIPT

<!--WEEK1:Build a Responsive Web Application for Registration Form, which contains User Name, Password, Date of Birth, Gender, Mail-id, Contact Number, Address and Submit Button--!>

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f4f7f8;
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      margin: 0;
    }

    .container {
      background: #fff;
      padding: 20px 30px;
      border-radius: 8px;
      box-shadow: 0 4px 8px rgba(0,0,0,0.1);
      max-width: 500px;
      width: 100%;
```

```
}

h2 {
  text-align: center;
  margin-bottom: 20px;
  color: #333;
}

.form-group {
  margin-bottom: 15px;
}

label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
  color: #555;
}

input, select, textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
  font-size: 14px;
}

textarea {
  resize: vertical;
}
```

```
.radio-inline {
  display: flex;
  gap: 20px; /* spacing between options */
  margin-top: 8px;
}

.radio-inline label {
  display: flex;
  align-items: center;
  font-size: 14px;
  cursor: pointer;
}

.radio-inline input[type="radio"] {
  margin-right: 6px;
}


    button {
      width: 100%;
      padding: 12px;
      background: #007BFF;
      color: #fff;
      border: none;
      border-radius: 5px;
      font-size: 16px;
      cursor: pointer;
    }
```

```
    button:hover {
      background: #0056b3;
    }

    /* Responsive Design */
    @media (max-width: 600px) {
      .container {
        padding: 15px;
      }
      h2 {
        font-size: 20px;
      }
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>Registration Form</h2>
    <form id="registrationForm">
      <div class="form-group">
        <label for="username">User Name</label>
        <input type="text" id="username" name="username" required>
      </div>

      <div class="form-group">
        <label for="password">Password</label>
        <input type="password" id="password" name="password" required>
      </div>
```

```
      <div class="form-group">
        <label for="dob">Date of Birth</label>
        <input type="date" id="dob" name="dob" required>
      </div>

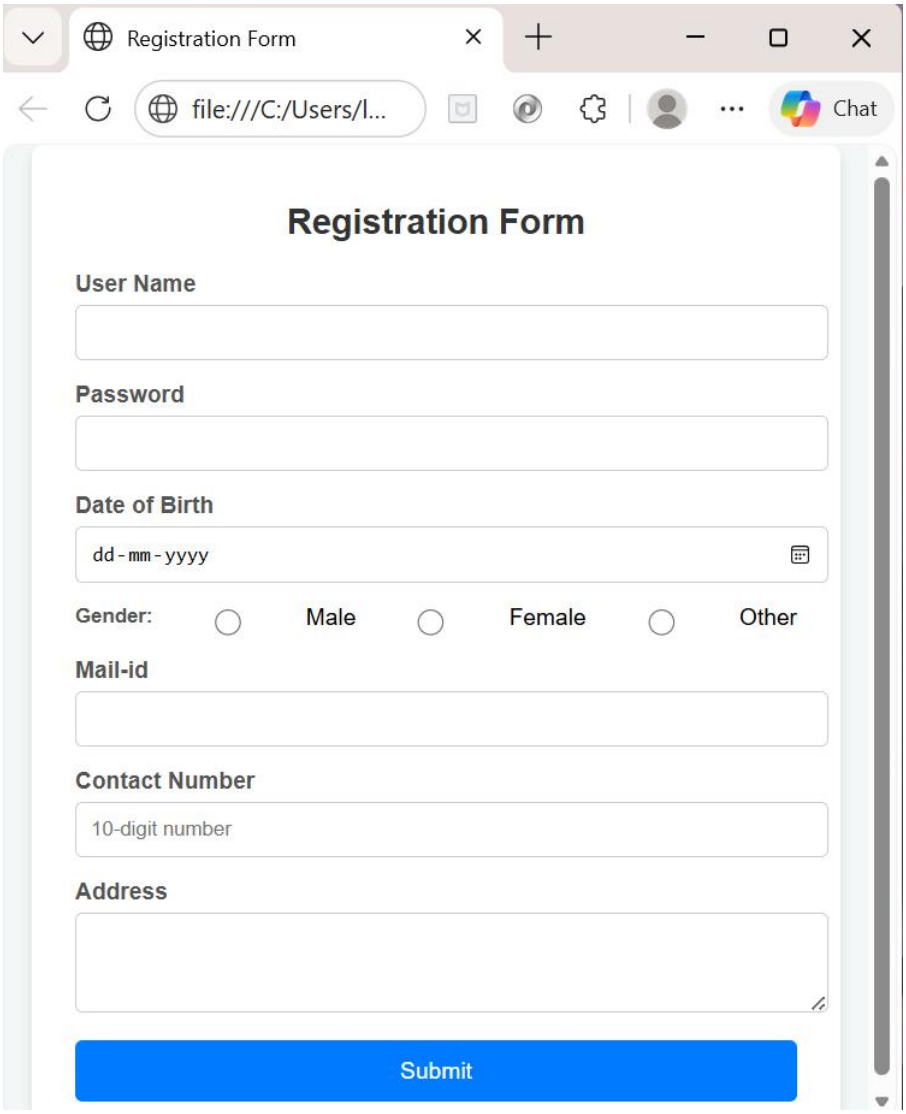
    <div class="form-group">
<div class="radio-inline">
         <label>Gender:</label>
         <input type="radio" name="gender" value="male" required>Male
         <input type="radio" name="gender" value="female" required>Female
         <input type="radio" name="gender" value="other" required>Other
</div> </div>
        <div class="form-group">
        <label for="email">Mail-id</label>
        <input type="email" id="email" name="email" required>
      </div>

      <div class="form-group">
        <label for="contact">Contact Number</label>
        <input type="tel" id="contact" name="contact" pattern="[0-9]{10}"
placeholder="10-digit number" required>
      </div>

      <div class="form-group">
        <label for="address">Address</label>
        <textarea id="address" name="address" rows="3" required></textarea>
      </div>

      <button type="submit">Submit</button>
    </form>
```

```
  </div>
  <script>
    document.getElementById("registrationForm").addEventListener("submit",
function(event) {
      event.preventDefault(); // Prevent form submission for demo
      alert("Form submitted successfully!");
    });
  </script>
</body>
</html>
```



Week2 : Implement a JavaScript Program to Validate Registration Page using Regular Expression.

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form with Validation</title>
</head>
<body>
  <h2>Registration Form</h2>
  <form id="registrationForm">
    <p>
      <label for="username">User Name:</label><br>
      <input type="text" id="username" name="username"
placeholder="Username" >
    </p>

    <p>
      <label for="password">Password:</label><br>
      <input type="password" id="password" name="password"
placeholder="Password">
    </p>

    <p>
      <label for="dob">Date of Birth:</label><br>
      <input type="date" id="dob" name="dob">
    </p>

    <p>
      <label>Gender:</label><br>
      <input type="radio" name="gender" value="male"> Male
      <input type="radio" name="gender" value="female" > Female
      <input type="radio" name="gender" value="other" > Other
    </p>

    <p>
```

```
    <label for="email">Mail-id:</label><br>
    <input type="email" id="email" name="email" placeholder="Email id">
  </p>

  <p>
    <label for="contact">Contact Number:</label><br>
    <input type="tel" id="contact" name="contact" placeholder="10-digit number">
  </p>

  <p>
    <label for="address">Address:</label><br>
    <textarea id="address" name="address" rows="3" placeholder="address"></textarea>
  </p>

  <p>
    <button type="submit">Submit</button>
  </p>
 </form>

 <script>
document.getElementById("registrationForm").addEventListener("submit", function(event) {
    event.preventDefault(); // Prevent default submission

    // Collect values
    const username = document.getElementById("username").value.trim();
    const password = document.getElementById("password").value.trim();
    const dob = document.getElementById("dob").value;
    const gender = document.querySelector('input[name="gender"]:checked');
    const email = document.getElementById("email").value.trim();
    const contact = document.getElementById("contact").value.trim();
    const address = document.getElementById("address").value.trim();

    // Regex patterns
```

```
    const usernameRegex = /^[A-Za-z0-9_]{3,15}$/;   // 3–15 chars, letters, numbers, underscore
    const passwordRegex = /^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{6,}$/; // min 6 chars, at least 1 letter & 1 number
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/; // basic email format
    const contactRegex = /^[0-9]{10}$/;             // exactly 10 digits

    // Validation checks
    if (!usernameRegex.test(username)) {
      alert("Invalid Username: Use 3–15 letters, numbers, or underscores.");
      return;
    }

    if (!passwordRegex.test(password)) {
      alert("Invalid Password: Must be at least 6 characters with letters and numbers.");
      return;
    }

    if (!dob) {
      alert("Please enter your Date of Birth.");
      return;
    }

    if (!gender) {
      alert("Please select your Gender.");
      return;
    }

    if (!emailRegex.test(email)) {
      alert("Invalid Email format.");
      return;
    }

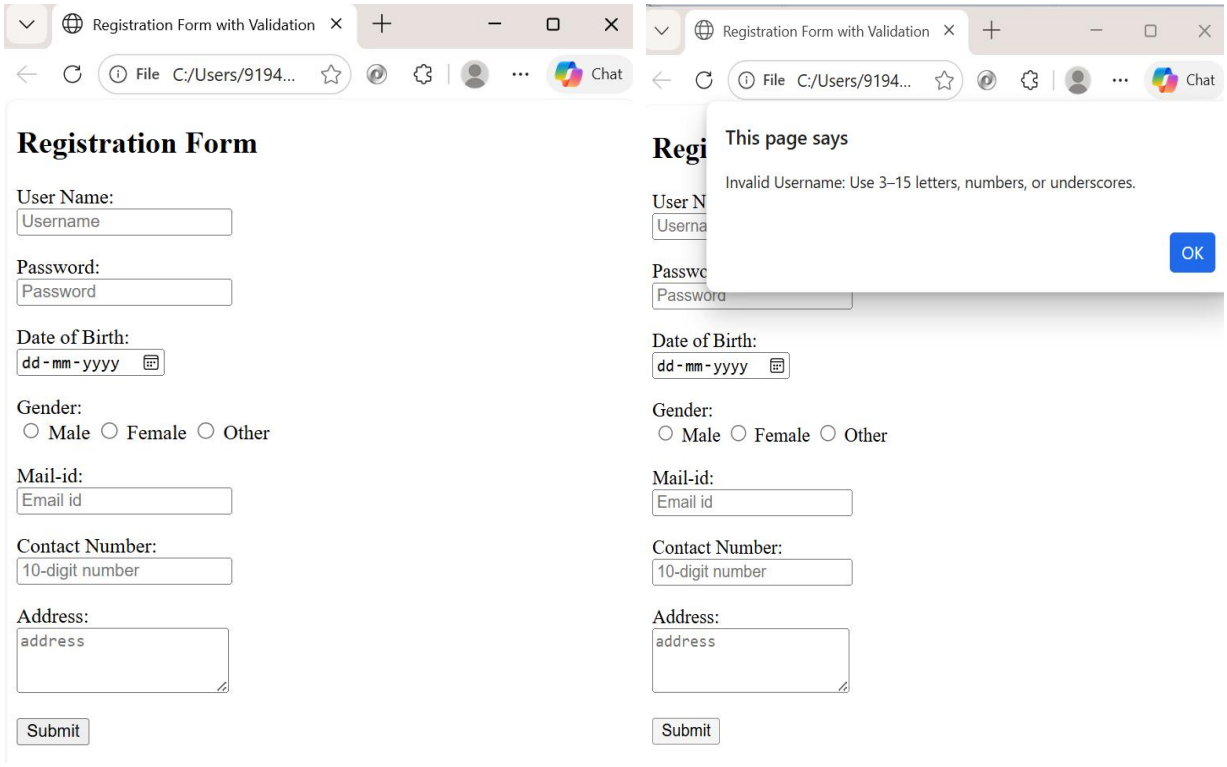
    if (!contactRegex.test(contact)) {
      alert("Invalid Contact Number: Must be exactly 10 digits.");
      return;
```

```
    }

    if (address.length < 5) {
      alert("Address must be at least 5 characters long.");
      return;
    }

    // If all validations pass
    alert("Form submitted successfully!");
  });
 </script>
</body>
</html>
```

OUTPUT:



WEEK3: Build a Responsive Web Application for Shopping Cart with Registration, Login, Catalog and Cart Pages using CSS3 features, Flex and Grid

PROGRAM:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>Shopping Cart</title>
    <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@4.0.0/dist/css/bootstrap.min.cs
s"crossorigin="anonymous">
        <link rel="stylesheet" href="assets/css/style.css">
    <link rel="icon" type="image/png" href="images/cart-icon.png">

</head>

<body>
   <header class="bg-dark text-white text-center py-3">
  <h1>Shopping Cart</h1>
</header>
      <!-- Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <a class="navbar-brand" href="#">Home</a>
  <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarNav">
    <span class="navbar-toggler-icon"></span>
  </button>

  <div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav ml-auto">
      <!-- Catalog -->
      <li class="nav-item">
        <a class="nav-link" href="#catalog">Catalog</a>
      </li>
      <!-- Login -->
      <li class="nav-item">
        <a class="nav-link" href="login.html">Login</a>
      </li>
      <!-- Registration -->
      <li class="nav-item">
```

```
      <a class="nav-link" href="registration.html">Register</a>
    </li>
    <!-- Cart -->
    <li class="nav-item">
      <a class="nav-link" href="#" data-toggle="modal" data-target="#cartModal">
        <img src="images/cart.jfif" alt="Cart" class="cart-icon"> Cart
      </a>
    </li>
   </ul>
  </div>
</nav>


      <!-- Main -->
      <div class="container">

         <br>

         <h3 class="text-center">Shopping Cart</h3>
         <hr>

         <div class="row">
            <div class="col-sm-12 col-md-6 col-xl-4">
               <figure class="card card-product zoom">
          <div class="img-wrap"><img src="images/dress1.jfif" class="rounded p-
1"></div>
                  <figcaption class="info-wrap">
                     <h4 class="title">Beautiful dress</h4>
                     <p class="desc">"Adorn yourself in the allure of timeless
beauty with our collection, where
                        each dress is a masterpiece of grace and charm."</p>
                     <div class="rating-wrap">
                        «<div class="label-rating">132 reviews</div>
                        <div class="label-rating">154 orders </div>
                        <br />
                        <small class="text-muted">&#9733; &#9733;
                           &#9733; &#9733; &#9734;</small>
                     </div> <!-- rating-wrap.// -->
                  </figcaption>
                  <div class=" bottom-wrap">
                     <button class="btn btn-primary add-to-cart" data-
name="Beautiful Dress" data-price="220">Add to Cart</button>

                     <div class="price-wrap h5">
```

```
                        <span class="price-new">$220</span> <del class="price-
old">$180</del>
                        <div>
                        </div>
                        <!--price-wrap.//-->
                    </div>
                    <!--bottom-wrap.//-->
                </figure>
            </div>
            <!--col//-->

            <div class="col-sm-12 col-md-6 col-xl-4">
                <figure class="card card-product zoom">
                    <div class="img-wrap"><img src="images/item-2.jpeg"
class="rounded p-1">
                    </div>

                    <figcaption class="info-wrap">
                        <h4 class="title">Elegant style</h4>
                        <p class="desc">"Elevate your wardrobe with an essence
through our collection, where
                            elegance becomes a signature style."</p>
                        <div class="rating-wrap">
                            <div class="label-rating">122 reviews</div>
                            <div class="label-rating">120 orders </div>
                            <br />
                            <small class="text-muted">&#9733; &#9733;
                                &#9733; &#9733; &#9734;</small>
                        </div>

                        <!-- rating-wrap.//-->
                    </figcaption>
                    <div class="bottom-wrap">
                         <button class="btn btn-primary add-to-cart" data-
name="Elegant Style" data-price="115">Add to Cart</button>

                        <div class="price-wrap h5">
                            <span class="price-new">$115</span> <del class="price-
old">$180</del>
                            <div>

                                <!--price-wrap.//-->
                            </div>
```

```
                        <!--bottom-wrap.//-->
                    </figure>
                </div>
                <!--col//-->
                <div class="col-sm-12 col-md-6 col-xl-4">

                    <figure class="card card-product zoom">
                        <div class="img-wrap"><img src="images/dress3.jfif"
class="rounded p-1"></div>

                        <figcaption class="info-wrap">
                            <h4 class="title">Modern fashion</h4>
                            <p class="desc">"Discover the epitome of modern fashion,
where style meets innovation in
                                every stitch and silhouette."</p>
                            <div class="rating-wrap">
                                <div class="label-rating">138 reviews</div>
                                <div class="label-rating">159 orders</div>
                                <br />

                                <small class="text-muted">&#9733;
&#9733;&#9733;&#9733;&#9733;&#9734</small>
                            </div>
                            <!-- rating-wrap.//-->
                        </figcaption>

                        <div class="bottom-wrap">
                          <button class="btn btn-primary add-to-cart" data-
name="Modern Fashion" data-price="680">Add to Cart</button>

                            <div class="price-wrap h5">
                                <span class="price-new">$680</span> <del class="price-
old">$980</del>
                            </div> <!-- price-wrap.//-->
                        </div> <!-- bottom-wrap.//-->
                    </figure>
                </div> <!-- col //-->
            </div> <!-- row.//-->

        <!-- Cart Modal -->
<div class="modal fade" id="cartModal" tabindex="-1" role="dialog">
  <div class="modal-dialog" role="document">
    <div class="modal-content">
      <div class="modal-header">
```

```
        <h5 class="modal-title">Your Cart</h5>
        <button type="button" class="close" data-
dismiss="modal">&times;</button>
      </div>
      <div class="modal-body">
        <ul id="cartItems" class="list-group"></ul>
        <p class="mt-3 font-weight-bold">Total: $<span
id="cartTotal">0</span></p>
      </div>
      <div class="modal-footer">
        <button class="btn btn-secondary" data-dismiss="modal">Close</button>
        <button class="btn btn-success">Checkout</button>
      </div>
    </div>
  </div>
</div>

<script src="https://code.jquery.com/jquery-3.2.1.slim.min.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@4.0.0/dist/js/bootstrap.min.js"><
/script>

<script>
  let cart = [];
  document.querySelectorAll('.add-to-cart').forEach(btn => {
    btn.addEventListener('click', () => {
      const name = btn.dataset.name;
      const price = parseFloat(btn.dataset.price);
      cart.push({name, price});
      updateCart();
      $('#cartModal').modal('show');
    });
  });

  function updateCart() {
    const cartItems = document.getElementById('cartItems');
    cartItems.innerHTML = '';
    let total = 0;
    cart.forEach(item => {
      const li = document.createElement('li');
      li.className = 'list-group-item';
      li.textContent = `${item.name} - $${item.price}`;
      cartItems.appendChild(li);
      total += item.price;
```

```
    });
    document.getElementById('cartTotal').textContent = total.toFixed(2);
  }
// Checkout function
document.querySelector('.btn-success').addEventListener('click', () => {
  if (cart.length === 0) {
    alert("Your cart is empty!");
    return;
  }

  let summary = "Order Summary:\n";
  cart.forEach(item => {
    summary += `${item.name} - $${item.price}\n`;
  });
  summary += `\nTotal: $${document.getElementById('cartTotal').textContent}`;

  alert(summary);   // Show order summary
  cart = [];        // Clear cart after checkout
  updateCart();     // Refresh cart display
  $('#cartModal').modal('hide'); // Close modal
});


</script>
</body>

</html>.
```

Style.css

```
body {

    padding-top: 80px;

    }

        /* product cart design */

    .show-cart li {

        display: flex;
```

```
}

.cart-container {
    position: relative;
    display: inline-block;
}
.cart-icon {
    width: 30px; /* Adjust the width and height according to your icon size */
    height: 30px;
    cursor: pointer;
}

/* Add a hover effect if desired */
.cart-container:hover .cart-icon {
    filter: brightness(1.2); /* Adjust the brightness for the hover effect */
}
.card {
margin-bottom: 20px;
}
.card-img-top {
width: 200px;
height: 200px;
align-self: center;
}
```

```
.card-product .img-wrap {
border-radius: 3px 3px 0 0;
overflow: hidden;
position: relative;
height: 220px;
text-align: center;
}
.card-product .img-wrap img {
max-height: 100%;
max-width: 100%;
object-fit: cover;
}
.card-product .info-wrap {
overflow: hidden;
padding: 15px;
border-top: 1px solid #eee;
}
.card-product .bottom-wrap {
padding: 15px;
border-top: 1px solid #eee;
}
.label-rating { margin-right:10px;
color: #333;
```

```
display: inline-block;
vertical-align: middle;
}


.card-product .price-old {
color: #999;

}
/* shopping cart counter */
.header{background:rgb(0, 178, 255);color:#fff;}
#lblCartCount {
    font-size: 12px;
    background: #ff0000;
     color: #fff;
     padding: 0 5px;
     vertical-align: top;
     margin-left: -10px; }
.badge {
    padding-right: 9px;
    -webkit-border-radius: 9px;
    -moz-border-radius: 9px;
    border-radius: 9px; }
    .label-warning[href],
```

```
    .badge-warning[href] {
        background-color: #c67605; }

/* shopping cart counter style */

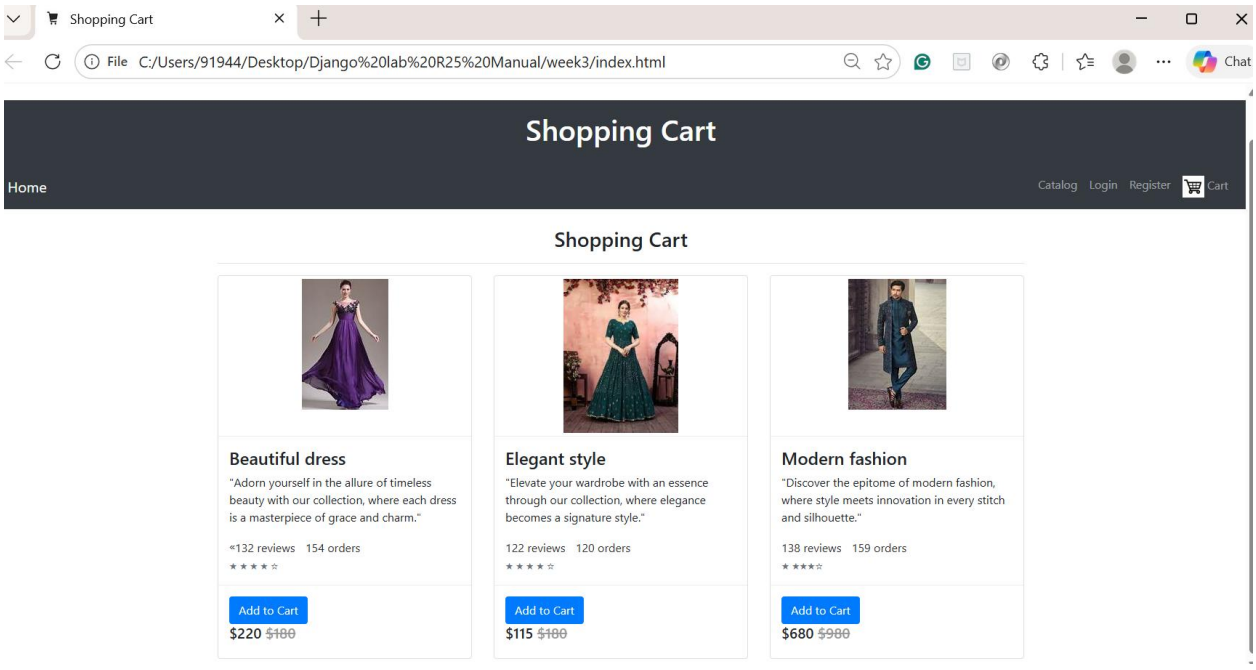
i.fa-shopping-cart {
    width:2.2em;
    text-align:center;
     vertical-align:middle;
    }
.fa-shopping-cart {
     margin-top: 0.2em;
     position: relative;
    }
.badge {
    font-size: .25em;
    display: block;
    position: absolute;
     top: -.70em;
     right: -.70em;
     width: 2.5em;
     height: 2.4em;
     line-height: 2em;
     border-radius: 50%;
```

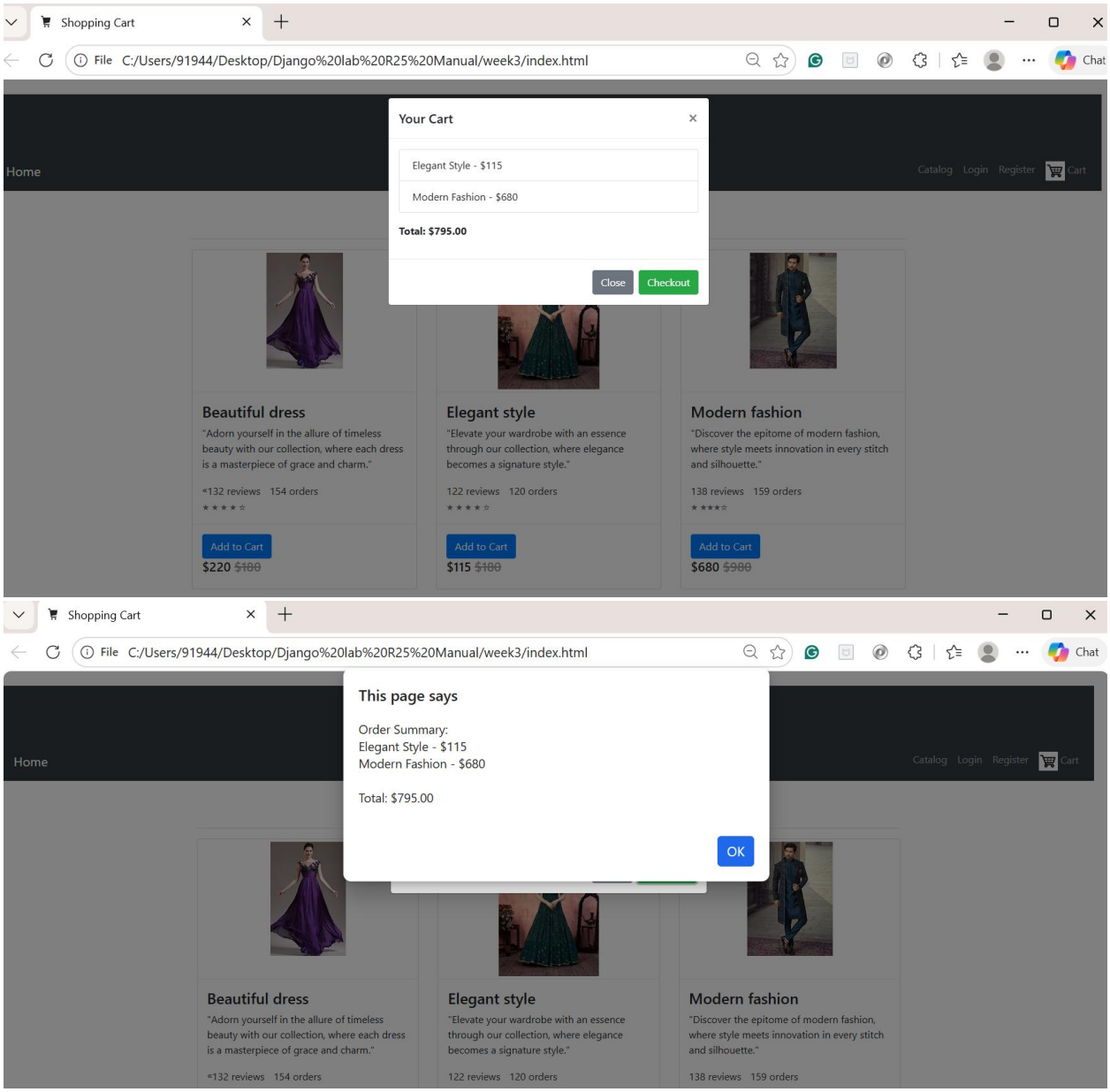
```
    text-align: center;
    color: #fff;
    background: rgba(207, 0, 15, 1);
   }
/* product zoom on hover */
.zoom, .zoom-sm {
transition: transform .2s;
}
.zoom:hover {
transform: scale(1.05);}
.zoom-sm:hover {
transform: scale(1.03);
}
/* shopping cart toast */
#product-toast {
visibility: hidden;
max-width: 50px;
height: 50px;
margin: auto;
background-color: #333;
color:#fff;
text-align:center;
border-radius:2px;
position:fixed;
```

```
    z-index:1;
    left:0;right:0;
    bottom:30px;
    font-size:17px;
    white-space:nowrap;
    }
    #product-toast #img{
    width: 70px;
    height:50px;
    float:left;
    padding-top:16px;
    padding-bottom:16px;
    box-sizing: border-box;
    background-color:#111;
    color:#fff;
    padding:16px;
    overflow:hidden;
    white-space:nowrap;
    }
    #product-toast.show{
    visibility:visible;
    -webkit-animation:fadein 0.5s,expand 0.5s 0.5s,stay 3s 1s,shrink 0.5s
2s,fadeout 0.5s 2.5s;
    animation:fadein 0.5s,expand 0.5s 0.5s,stay 3s 1s,shrink 0.5s 4s,fadeout
0.5s 4.5s;
```

```
}
@keyframes fadein {
from {bottom: 0; opacity: 0;}
to {bottom: 30px; opacity: 1;}
}
@-webkit-keyframes expand {
from {min-width: 50px}
to {min-width: 350px}
}
@keyframes expand {
from {min-width: 50px}
to {min-width: 350px}
}
@-webkit-keyframes stay {
from {min-width: 350px}
to {min-width: 350px}
}
@keyframes stay {
from {min-width: 350px}
to {min-width: 350px}
}
@-webkit-keyframes shrink {
from {min-width: 350px;}
to {min-width: 50px;}
```

```
}
@keyframes shrink {
from {min-width: 350px;}
to {min-width: 50px;}
}
@-webkit-keyframes fadeout {
from {bottom: 30px; opacity: 1;}
to {bottom: 60px; opacity: 0;}
}
@keyframes fadeout {
from {bottom: 30px; opacity: 1;}
to {bottom: 60px; opacity: 0;}
}
```




Shopping Cart
File C:/Users/91944/Desktop/Django%20lab%20R25%20Manual/week3/index.html
Chat
Your Cart
Elegant Style - $115
Modern Fashion - $680
Total: $795.00
Close
Checkout
Home
Catalog
Login
Register
Cart
Beautiful dress
"Adorn yourself in the allure of timeless beauty with our collection, where each dress is a masterpiece of grace and charm."
132 reviews 154 orders
Add to Cart
$220 $180
Elegant style
"Elevate your wardrobe with an essence through our collection, where elegance becomes a signature style."
122 reviews 120 orders
Add to Cart
$115 $180
Modern fashion
"Discover the epitome of modern fashion, where style meets innovation in every stitch and silhouette."
138 reviews 159 orders
Add to Cart
$680 $980
Shopping Cart
File C:/Users/91944/Desktop/Django%20lab%20R25%20Manual/week3/index.html
Chat
This page says
Order Summary:
Elegant Style - $115
Modern Fashion - $680
Total: $795.00
OK
Home
Catalog
Login
Register
Cart
Beautiful dress
"Adorn yourself in the allure of timeless beauty with our collection, where each dress is a masterpiece of grace and charm."
132 reviews 154 orders
Elegant style
"Elevate your wardrobe with an essence through our collection, where elegance becomes a signature style."
122 reviews 120 orders
Modern fashion
"Discover the epitome of modern fashion, where style meets innovation in every stitch and silhouette."
138 reviews 159 orders

WEEK4: Basics of Django Framework & Installation of required Software's

Step1: install Python (latest version)

python –version

pip –version

pip install virtualenv or venv

Create parent directory to save all Django projects

mkdir my_django_project

cd my_django_project

Step2: create virtual Enviroment

C:\>python -m venv week4

Next, Activate the enviroment

.\env\Scripts\activate

Or

C:\> cd week4

C:\week4\>cd Scripts

C:\> week4\Scripts**.\activate.bat or .ps1(powershell)**

Step3 : install django

pip install django

to know the version

django-admin --version

step4:create Django project

django-admin startproject **website** ← **project name**

cd website

to check successfully installed Django or not runserver

python manage.py runserver

then get Django welcome page – means successfully django installed

step4: create simple application -

step4.1 To Create app

#to create app –

python manage.py startapp myapp

Create these files separately

HTML- index.html

CSS-style.css

Javascript- script.js

Step 4.2 create subfolders in myapp

static – add CSS and js files

templates –HTML files

Project Structure

website/

├── myapp/

│   ├── templates/

│   │   └── myapp/

│   │   └── index.html

```
|   ├── static/
|   |   └── myapp/
|   |       |   └── style.css
|   |       |   └── script.js
```

In Templates\myapp\Index.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="{% static 'myapp/style.css' %}">
  <script src="{% static 'myapp/script.js' %}"></script>
</head>
<body>
  <h1>Welcome to My Django App</h1>
  <button onclick="showMessage()">Click Me</button>
  <p id="output"></p>
</body>
</html>
```

In Static\myapp\script.js

```
function showMessage() {
  document.getElementById("output").innerText = "Hello from JavaScript!";
}
```

In Static\myapp\style.css

```
body {
  font-family: Arial;
  background-color: #f0f0f0;
}
button {
  padding: 10px;
  background-color: #007bff;
  color: white;
  border: none;
}
```

Django Templates

Templates are HTML files with Django's own template language (DTL) embedded. They allow you to:

- Insert dynamic data using {{ variable }}
- Control logic with {% tags %} like loops and conditionals
- Extend base layouts using {% extends %} and {% block %}

Step5 :Configure Template directory in settings.py

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],  # Your custom templates folder
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

For Static directory configure in settings.py

```
STATIC_URL = '/static/'
```

```
STATICFILES_DIRS = [

    BASE_DIR /  'static',

]
```

OR

```
STATICFILES_DIRS = [

    BASE_DIR / 'myapp' / 'static',

]
```

Step 6: Add myapp in installedApps settings.py

```
#Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp',
]
```

Step7: In Django, views.py and urls.py are essential for handling how your web app responds to user requests.

Views.py

This file contains **view functions or classes** that define what happens when a user visits a specific URL.

Purpose: It processes requests and returns responses.

- **Common tasks**:
- Fetch data from the database
- Render templates (HTML pages)
- Handle form submissions
- Return JSON for APIs

views.py

```
from django.shortcuts import render

def index(request):
    return render(request, 'myapp/index.html')
```

urls.py The Routing System

This file maps **URL patterns** to the corresponding view functions or classes.

- **Purpose**: It connects user requests (like /home) to the right logic in views.py.
- **Structure**: You define urlpatterns using Django's path() or re_path().

Example:

If not exists , we need to create urls.py in myapp

Myapp/urls.py- This file is **specific to one app** and defines the routes for that app's views

```
# urls.py

from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
]
```

You can also include app-specific urls.py files in your project-level urls.py using include():

website/urls.py - This file is the **central URL dispatcher** for your entire Django project

```
# project-level urls.py

from django.contrib import admin

from django.urls import path, include

urlpatterns = [

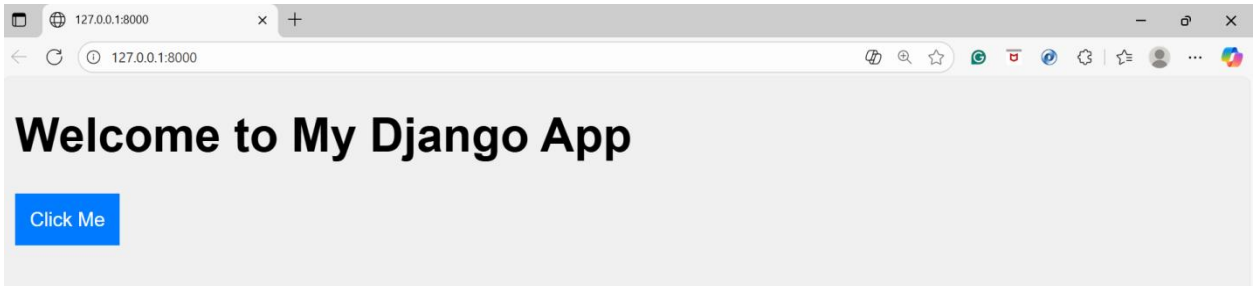
    path('admin/', admin.site.urls),
```

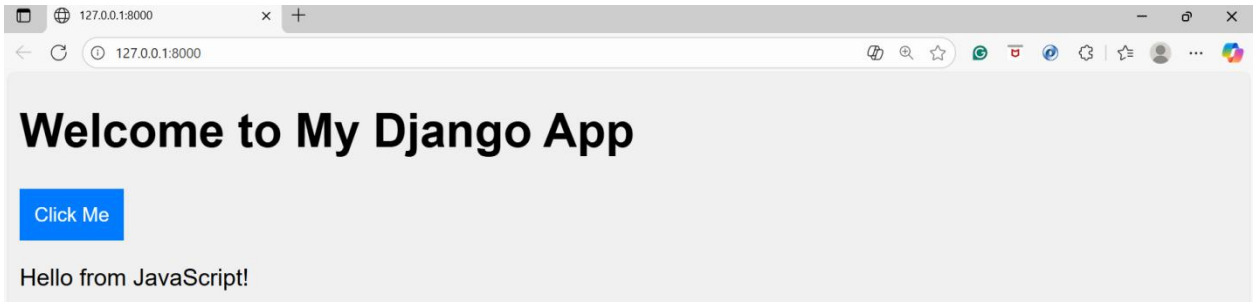
```
    path('', include('myapp.urls')),
]
```

Step9: Run application/project

python manage.py runserver

OUTPUT:





WEEK5: Create a shopping cart web application and add a customized admin page for product catalog management for Experiment 3

Project Structure:

```
shopping_cart/
├── manage.py
├── shopping_cart/        # Project settings
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── shop/                 # Main app
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── views.py
    ├── urls.py
    ├── forms.py
    └── templates/
        ├── base.html
        ├── registration.html
        ├── login.html
        ├── catalog.html
        └── cart.html
```

Step1:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'shop',
]
LOGIN_REDIRECT_URL = 'catalog'
LOGOUT_REDIRECT_URL = 'login'
```

Step2:

```
#views.py
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import login
#from django.contrib.auth import logout
from django.shortcuts import redirect
from django.contrib.auth.decorators import login_required
from .forms import RegistrationForm, CartItemForm
from .models import Product, CartItem

def register(request):
    if request.method == 'POST':
        form = RegistrationForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            return redirect('catalog')
    else:
        form = RegistrationForm()
    return render(request, 'registration.html', {'form': form})

@login_required
def catalog(request):
    products = Product.objects.all()
    return render(request, 'catalog.html', {'products': products})

@login_required
def add_to_cart(request, product_id):
```

```
    product = get_object_or_404(Product, id=product_id)
    cart_item, created = CartItem.objects.get_or_create(user=request.user,
product=product)
    if not created:
        cart_item.quantity += 1
    cart_item.save()
    return redirect('cart')

@login_required
def cart(request):
    items = CartItem.objects.filter(user=request.user)
    total = sum(item.subtotal() for item in items)
    return render(request, 'cart.html', {'items': items, 'total': total})

@login_required
def update_cart(request, item_id):
    item = get_object_or_404(CartItem, id=item_id, user=request.user)
    if request.method == 'POST':
        form = CartItemForm(request.POST, instance=item)
        if form.is_valid():
            form.save()
    return redirect('cart')
#def user_logout(request):


    logout(request)   # clears the session
    return redirect('catalog')  # redirect to catalog or homepage

#models.py
from django.db import models
from django.contrib.auth.models import User

class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=8, decimal_places=2)
    description = models.TextField(blank=True)

    def __str__(self):
        return self.name

class CartItem(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    product = models.ForeignKey(Product, on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField(default=1)
```

```
    def subtotal(self):
        return self.product.price * self.quantity

#forms.py
from django import forms
from django.contrib.auth.models import User
from django.contrib.auth.forms import UserCreationForm
from .models import CartItem

class RegistrationForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ['username', 'email', 'password1', 'password2']

class CartItemForm(forms.ModelForm):
    class Meta:
        model = CartItem
        fields = ['quantity']


# admin.py
from django.contrib import admin
from .models import Product, CartItem

@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ('name', 'price', 'description')
    search_fields = ('name',)

@admin.register(CartItem)
class CartItemAdmin(admin.ModelAdmin):
    list_display = ('user', 'product', 'quantity', 'subtotal')


shop/urls.py
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views

urlpatterns = [
    path('register/', views.register, name='register'),
```

```
    path('catalog/', views.catalog, name='catalog'),
    path('cart/', views.cart, name='cart'),
    path('add/<int:product_id>/', views.add_to_cart, name='add_to_cart'),
    path('update/<int:item_id>/', views.update_cart, name='update_cart'),
    path('login/', auth_views.LoginView.as_view(template_name='login.html'),
name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),

]
```

Shopping_cart/Urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('shop.urls')),

]
```

In shop/templates folder

Base.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Shopping Cart</title>
</head>
<body>
    <nav>
        <a href="{% url 'catalog' %}">Catalog</a> |
        <a href="{% url 'cart' %}">Cart</a> |
        {% if user.is_authenticated %}
            <a href="{% url 'logout' %}">Logout</a>
        {% else %}
            <a href="{% url 'login' %}">Login</a> |
            <a href="{% url 'register' %}">Register</a>
        {% endif %}
    </nav>
    <hr>
    {% block content %}{% endblock %}
</body>
</html>
```

Registration.html

```
{% extends 'base.html' %}
{% block content %}
```

```
<h2>Create an Account</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Register</button>
</form>
<p>Already have an account? <a href="{% url 'login' %}">Login here</a>.</p>
{% endblock %}
```

Login.html

```
{% extends 'base.html' %}
{% block content %}
<h2>Login</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Login</button>
</form>
<p>Don't have an account yet? <a href="{% url 'register' %}">Register
here</a>.</p>
{% endblock %}
```

Catalog.html

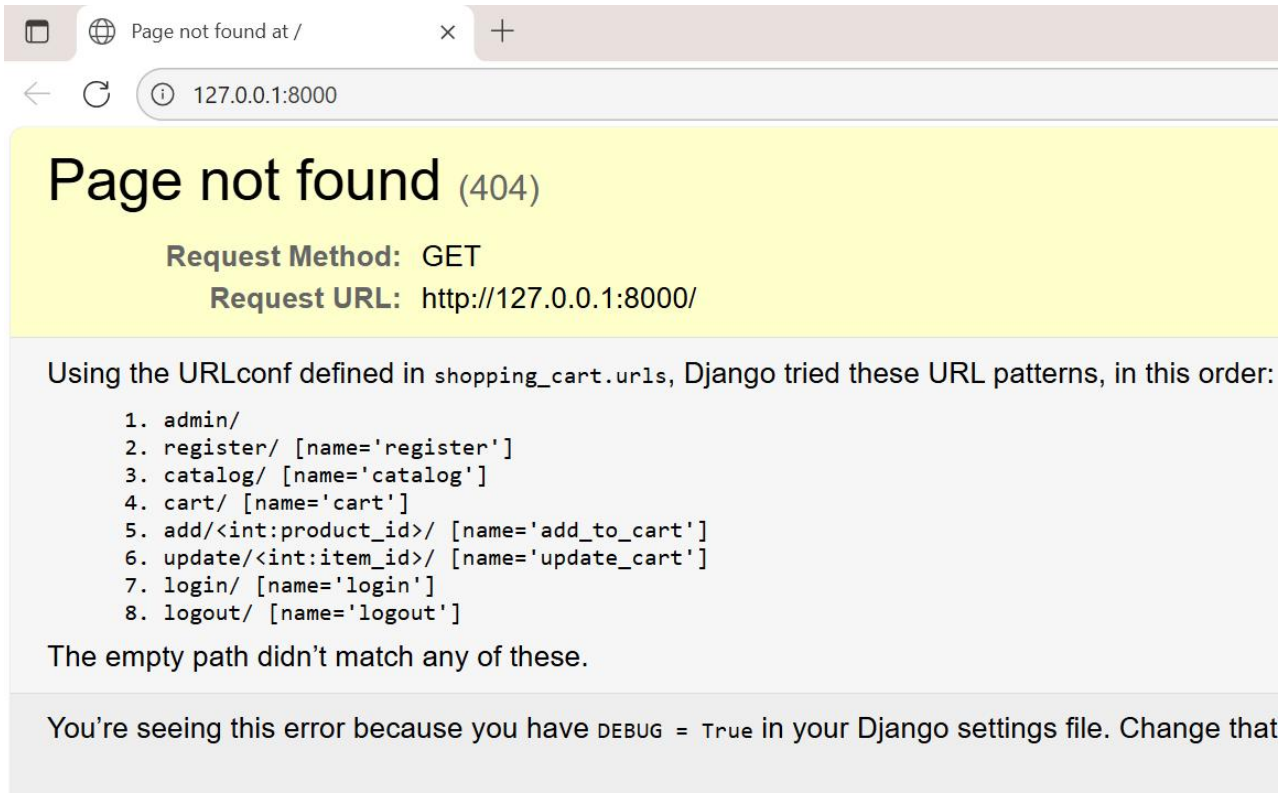
```
{% extends 'base.html' %}
{% block content %}
<h2>Product Catalog</h2>
<ul>
    {% for product in products %}
        <li>{{ product.name }} - ${{ product.price }}
            <a href="{% url 'add_to_cart' product.id %}">Add to Cart</a>
        </li>
    {% endfor %}
</ul>
{% endblock %}
```

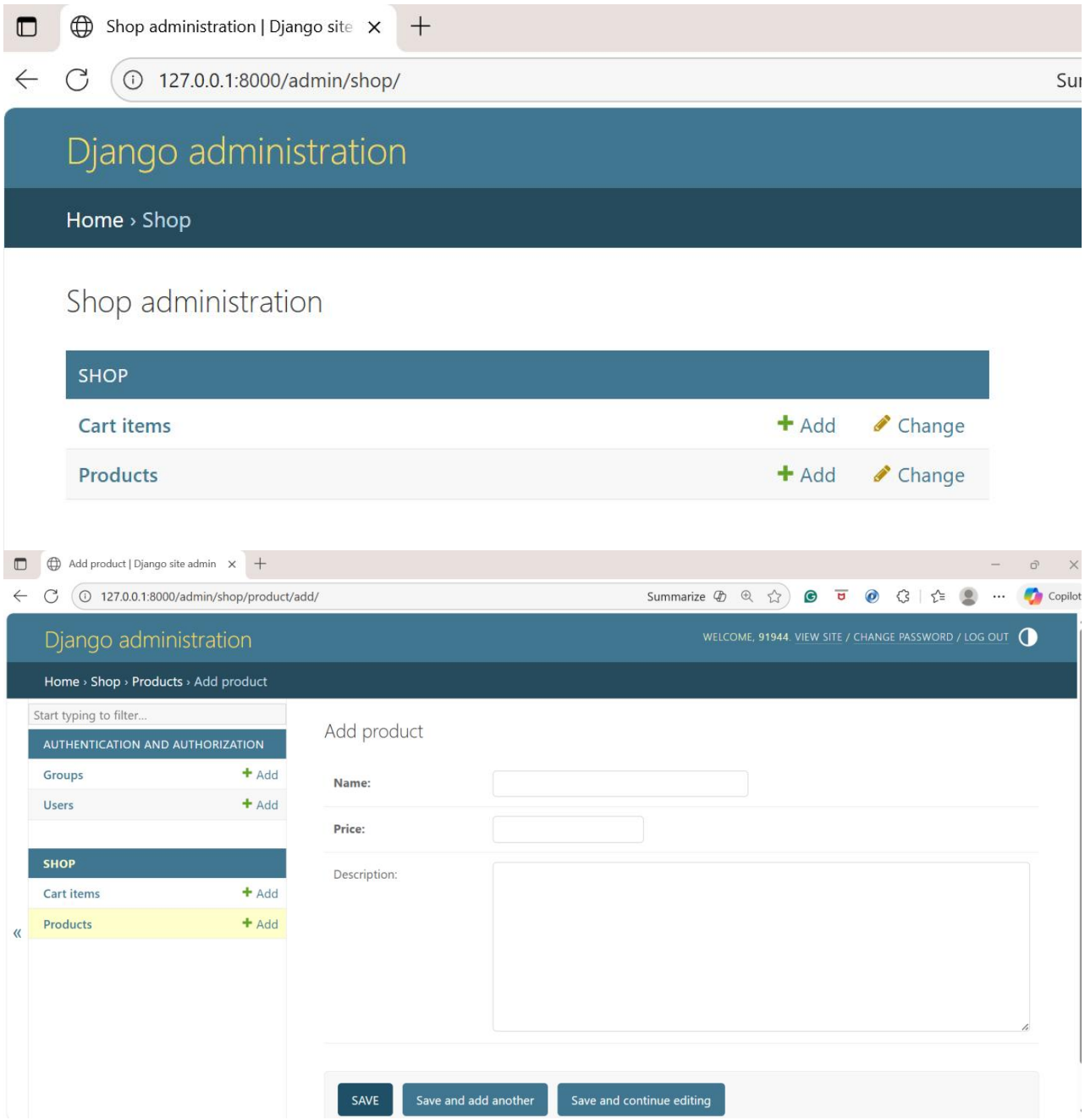
Cart.html

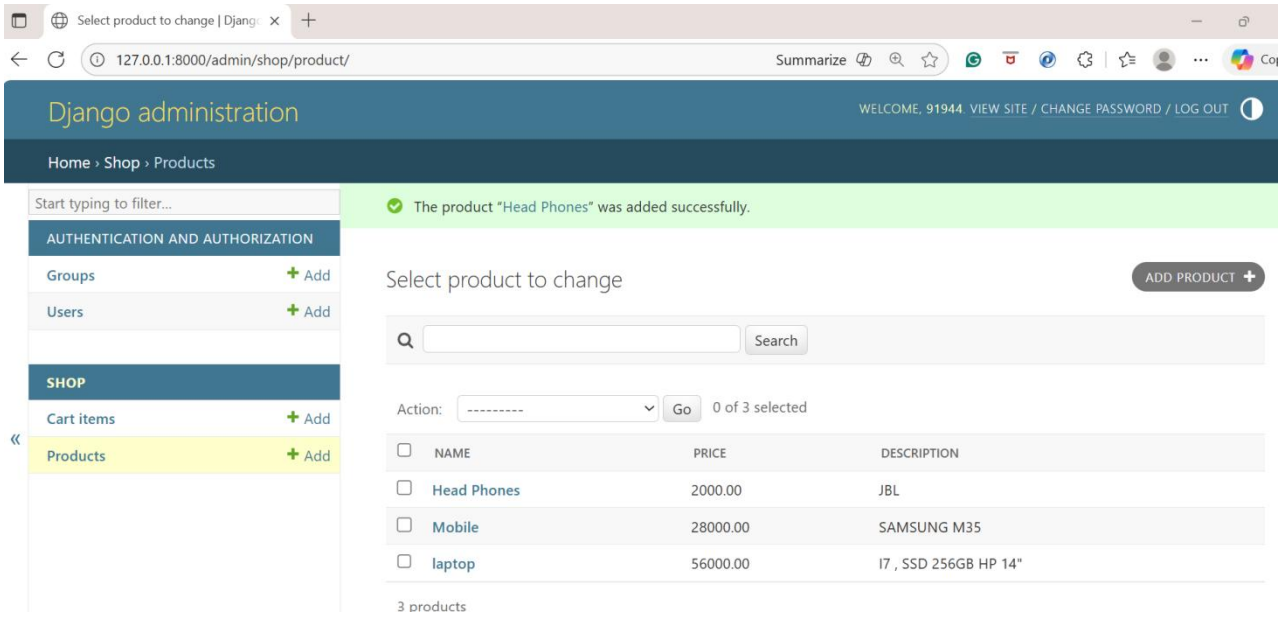
```
{% extends 'base.html' %}
{% block content %}
<h2>Your Cart</h2>
<ul>
    {% for item in items %}
        <li>
            {{ item.product.name }} (x{{ item.quantity }}) - ${{ item.subtotal }}
            <form method="post" action="{% url 'update_cart' item.id %}">
                {% csrf_token %}
```

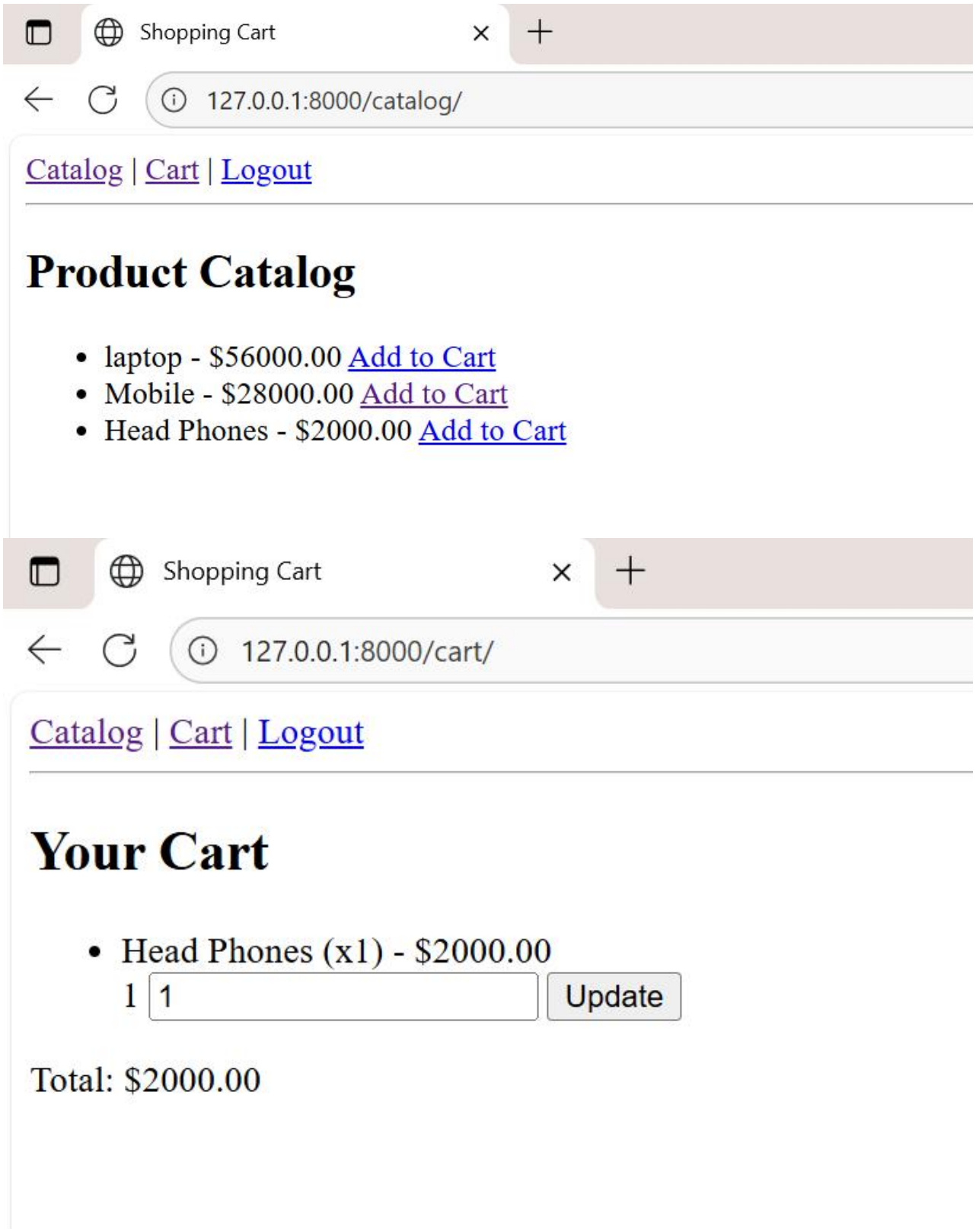
```
            {{ item.quantity }}
            <input type="number" name="quantity" value="{{ item.quantity }}"
min="1">
            <button type="submit">Update</button>
          </form>
        </li>
    {% endfor %}
</ul>
<p>Total: ${{ total }}</p>
{% endblock %}
```

Commands
python manage.py makemigrations
python manage.py migrate
Create Superuser:
python manage.py createsuperuser
Run Application:
python manage.py runserver

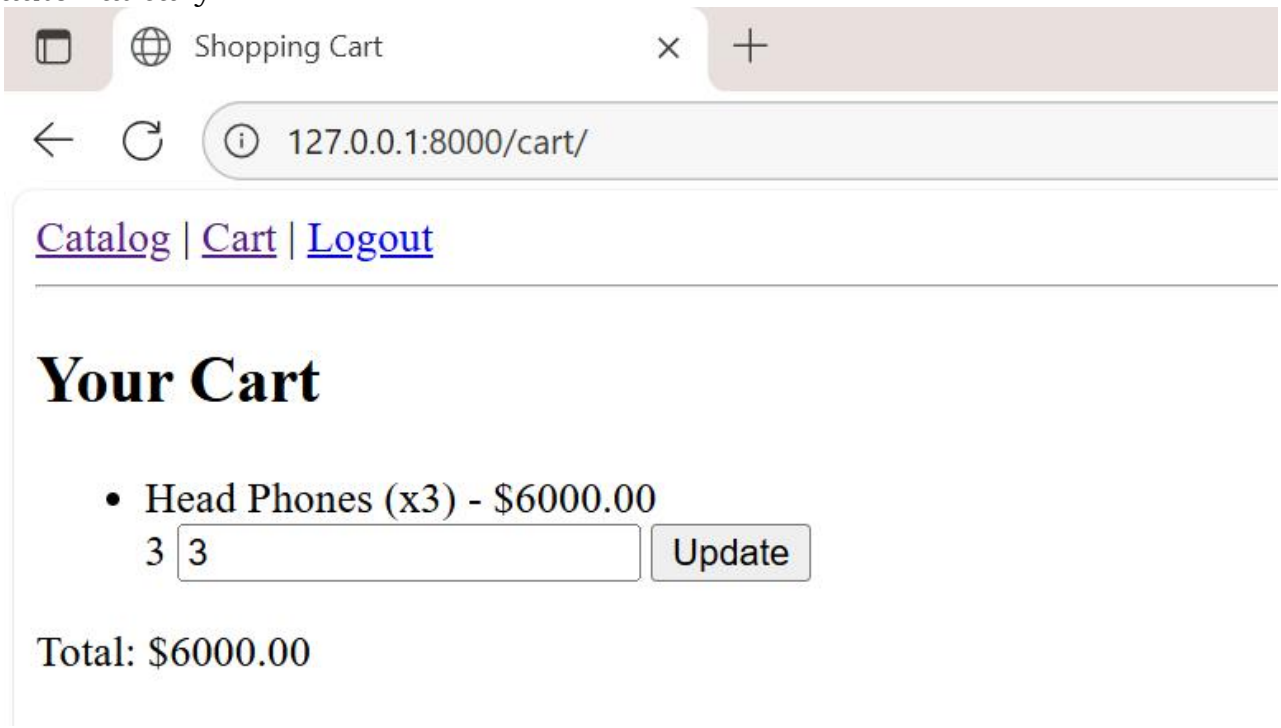



Shop administration | Django site
127.0.0.1:8000/admin/shop/
Django administration
Home › Shop
Shop administration
SHOP
Cart items
Add
Change
Products
Add
Change
Add product | Django site admin
127.0.0.1:8000/admin/shop/product/add/
Summarize
Copilot
Django administration
WELCOME, 91944. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Shop › Products › Add product
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups
Add
Users
Add
SHOP
Cart items
Add
Products
Add
Add product
Name:
Price:
Description:
SAVE
Save and add another
Save and continue editing


Select product to change | Django
127.0.0.1:8000/admin/shop/product/
Summarize
Django administration
WELCOME, 91944. VIEW SITE / CHANGE PASSWORD / LOG OUT
Home › Shop › Products
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
SHOP
Cart items + Add
Products + Add
The product "Head Phones" was added successfully.
Select product to change
ADD PRODUCT +
Search
Action: --------- Go 0 of 3 selected
NAME
PRICE
DESCRIPTION
Head Phones 2000.00 JBL
Mobile 28000.00 SAMSUNG M35
laptop 56000.00 I7 , SSD 256GB HP 14"
3 products


Shopping Cart
127.0.0.1:8000/catalog/
Catalog | Cart | Logout
Product Catalog
laptop - $56000.00 Add to Cart
Mobile - $28000.00 Add to Cart
Head Phones - $2000.00 Add to Cart
Shopping Cart
127.0.0.1:8000/cart/
Catalog | Cart | Logout
Your Cart
Head Phones (x1) - $2000.00
1
1
Update
Total: $2000.00

Page not found at /update/3 add more items to cart redirected to cart page automatically



WEEK6: Demonstrate Cache, Session Management for Student Login Application.

Project Structure

```
student_login_project/
│
    ├── manage.py
│
├── student_login_project/       # Project configuration
│   ├── __init__.py
│   ├── settings.py             # Cache + session settings
│   ├── urls.py                 # Project-level URL routing
│   ├── asgi.py
│   └── wsgi.py
│
├── studentlogin/                  # App for student login
│   ├── __init__.py
│   ├── admin.py                # Register Student model
│   ├── apps.py
│   ├── models.py               # Student model (username, password)
│   ├── views.py                # Login, logout, dashboard (cache + session + cookie)
│   ├── urls.py                 # App-specific routes
│   ├── forms.py                # (Optional) Django form for login
│   ├── middleware.py           # (Optional) custom session/cookie checks
│   └── templates/
│       ├── login.html       # Login form
│       └── dashboard.html    # Dashboard with logout link
│
└── db.sqlite3                  # Default database
```

Step1: Create a environment
Step2: Create Django Project –student_login_project
Step3: Create App-studentlogin

Step4:
Settings.py
Application definition

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
```

```
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'studentlogin',
]

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': ['BASE_DIR /TEMPLATES'],
         .....

CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.locmem.LocMemCache",
        "LOCATION": "student-cache",
    }
}


# Sessions are enabled by default
SESSION_ENGINE = "django.contrib.sessions.backends.db"
```

Step5:
Create a Model-Models.py

```
# models.py
from django.db import models
from django.contrib.auth.hashers import make_password, check_password

class Student(models.Model):
    username = models.CharField(max_length=50, unique=True)
    password = models.CharField(max_length=128)  # hashed password

    def save(self, *args, **kwargs):
        self.password = make_password(self.password)
        super().save(*args, **kwargs)

    def check_password(self, raw_password):
        return check_password(raw_password, self.password)

    def __str__(self):
        return self.username
```

Step6: Admin Module

```
#admin.py
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ("username",)
```

Step7:

```
# views.py
from django.shortcuts import render
from django.http import HttpResponse
from django.core.cache import cache
from .models import Student

def login_view(request):
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")

        # 1 Check cache first
        student = cache.get(username)
        if not student:
            try:
                student = Student.objects.get(username=username)
                cache.set(username, student, timeout=300)  # cache for 5 mins
            except Student.DoesNotExist:
                return HttpResponse("Invalid username")

        # 2 Validate password
        if student.check_password(password):
            # 3 Create session
            request.session["student_id"] = student.id
            request.session["username"] = student.username

            # 4 Create cookie
            response = HttpResponse(f"Welcome {student.username}!")
            response.set_cookie("student_username", student.username,
max_age=3600)
            return response
        else:
            return HttpResponse("Invalid password")
```

```
    return render(request, "login.html")

#dashboard
def dashboard_view(request):
    if "student_id" in request.session:
        return HttpResponse(f"Welcome {request.session['username']}! <a
href='/logout/'>Logout</a>")
    else:
        return HttpResponse("Please login first.")


#logout
def logout_view(request):
    # Clear session
    request.session.flush()

    # Clear cookie
    response = HttpResponse("Logged out successfully!")
    response.delete_cookie("student_username")
    return response
```

Step8:
AppLevel

```
# studentlogin/urls.py(AppLevel)

from django.urls import path
from . import views

urlpatterns = [
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('dashboard/', views.dashboard_view, name='dashboard'),  # optional
]


# student_login_project/urls.py(project level)
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('studentlogin.urls')),  # include app routes
]
```

Step9:
Create Templates folder (studentlogin/templates)
Login.html

```
<form method="post">
  {% csrf_token %}
  <label>Username:</label>
  <input type="text" name="username" required>
  <label>Password:</label>
  <input type="password" name="password" required>
  <button type="submit">Login</button>
</form>
```

Dashboard.html

```
<h2>Welcome {{ request.session.username }}</h2>
<p>You are logged in as student ID: {{ request.session.student_id }}</p>

<!-- Logout link -->
<a href="{% url 'logout' %}">Logout</a>
```

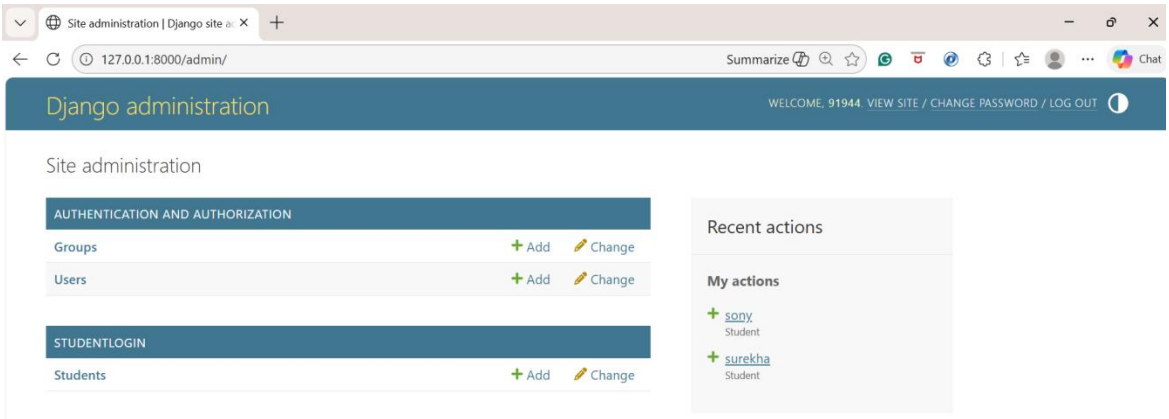
Commands:

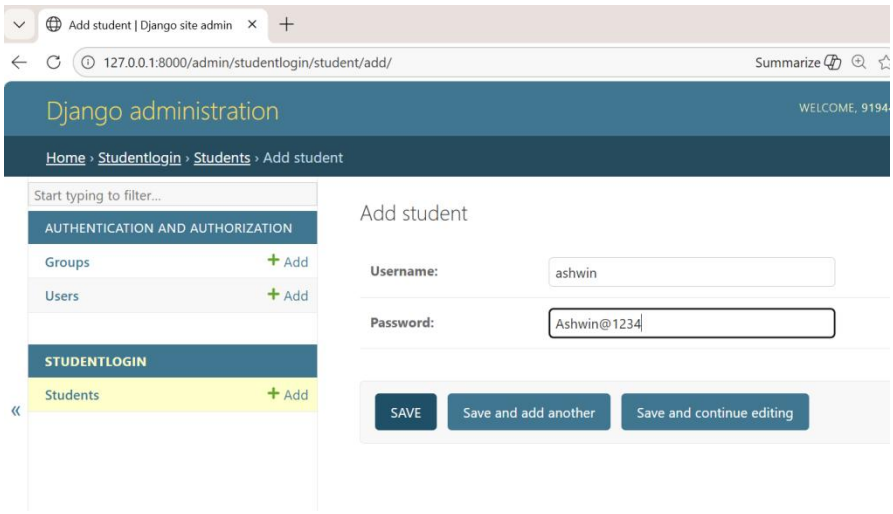
```
Python manage.py makemigrations
Python manage.py migrate

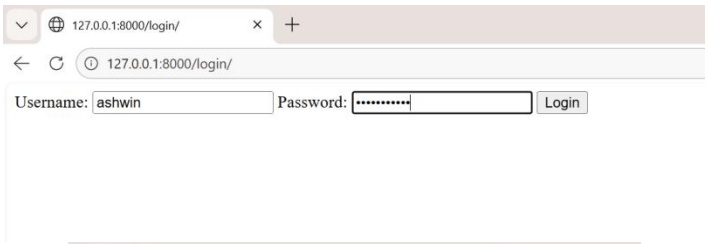
#Create superuser
python manage.py createsuperuser
Username: admin
Email address: admin@example.com
Password: ********
Password (again): ********
Superuser created successfully.

python manage.py runserver
```

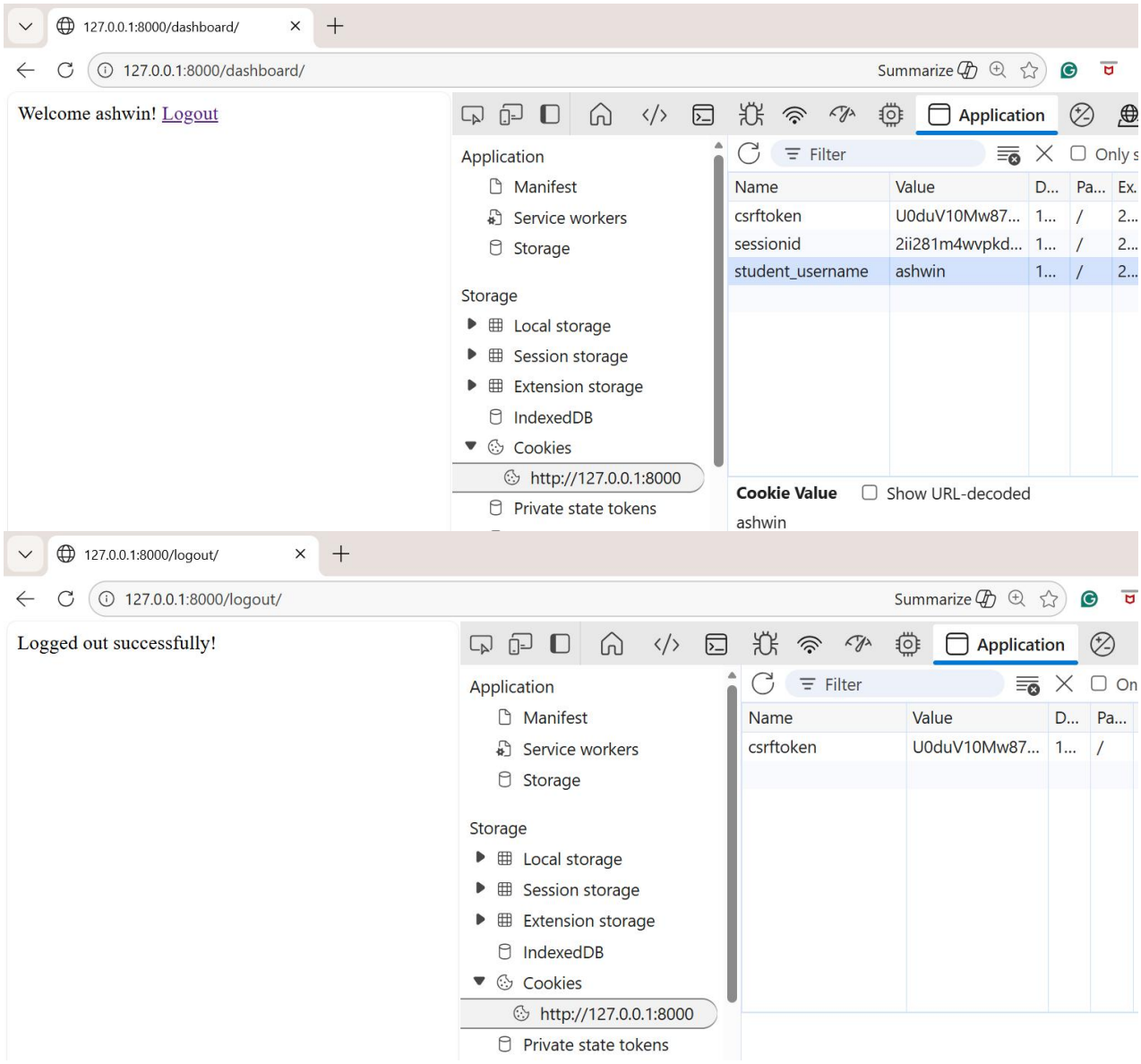
Visit:
http://127.0.0.1:8000/admin/


Site administration | Django site a
127.0.0.1:8000/admin/
Summarize
Chat
Django administration
WELCOME, 91944. VIEW SITE / CHANGE PASSWORD / LOG OUT
Site administration
AUTHENTICATION AND AUTHORIZATION
Groups + Add Change
Users + Add Change
STUDENTLOGIN
Students + Add Change
Recent actions
My actions
+ sony
Student
+ surekha
Student


Add student | Django site admin
127.0.0.1:8000/admin/studentlogin/student/add/
Summarize
Django administration
WELCOME, 9194
Home › Studentlogin › Students › Add student
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
STUDENTLOGIN
Students + Add
Add student
Username: ashwin
Password: Ashwin@1234
SAVE
Save and add another
Save and continue editing


127.0.0.1:8000/login/
127.0.0.1:8000/login/
Username: ashwin
Password:
Login


127.0.0.1:8000/login/
127.0.0.1:8000/login/
Welcome ashwin!


127.0.0.1:8000/dashboard/
127.0.0.1:8000/dashboard/
Summarize
Welcome ashwin! Logout
Application
Application
Manifest
Service workers
Storage
Storage
Local storage
Session storage
Extension storage
IndexedDB
Cookies
http://127.0.0.1:8000
Private state tokens
Filter
Only s
Name
Value
D...
Pa...
Ex.
csrftoken
U0duV10Mw87...
1...
/
2...
sessionid
2ii281m4wvpkd...
1...
/
2...
student_username
ashwin
1...
/
2...
Cookie Value
Show URL-decoded
ashwin
127.0.0.1:8000/logout/
127.0.0.1:8000/logout/
Summarize
Logged out successfully!
Application
Application
Manifest
Service workers
Storage
Storage
Local storage
Session storage
Extension storage
IndexedDB
Cookies
http://127.0.0.1:8000
Private state tokens
Filter
On
Name
Value
D...
Pa...
csrftoken
U0duV10Mw87...
1...
/

WEEK7: Create a Quiz Application using Django Framework along with SQLite3.

Steps:

#Create an Environment

python –m venv week7

cd week7/scripts/activate

Install Django if not already

pip install django

Create a new Django project

django-admin startproject quiz_app

cd quiz_app

Create a quiz application inside the project

python manage.py startapp quiz

```
quiz_app/                  # Root project folder
│
├── manage.py               # Django command-line utility
│
├── quiz_app/               # Project settings & configuration
│   ├── __init__.py
│   ├── settings.py         # Global settings (DB, installed apps, etc.)
│   ├── urls.py             # Root URL routing
│   ├── asgi.py             # ASGI entry point
```

```
│   └── wsgi.py              # WSGI entry point
│
├── quiz/                    # Our quiz application
│   ├── __init__.py
│   ├── admin.py             # Register models for Django admin
│   ├── apps.py              # App configuration
│   ├── models.py            # Database models (Question, Choice)
│   ├── views.py             # Business logic (quiz & results)
│   ├── urls.py              # App-specific URL routing
│   ├── tests.py             # Unit tests
│   ├── migrations/          # Auto-generated DB migrations
│   │   └── __init__.py
│   └── templates/quiz/      # HTML templates
│       ├── quiz.html        # Quiz page
│       └── result.html      # Results page
│
├── db.sqlite3               # SQLite3 database file
└── static/                  # (Optional) static files (CSS, JS, images)
```

Settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'quiz',  # our app
]
DEBUG = True   # Keep ON during development
```

```
ALLOWED_HOSTS = ['127.0.0.1', 'localhost']
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / "templates"],  # global templates folder
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / "static"]


LOGIN_REDIRECT_URL = '/'
LOGOUT_REDIRECT_URL = '/'

LANGUAGE_CODE = 'en-in'
TIME_ZONE = 'Asia/Kolkata'
USE_I18N = True
USE_TZ = True
#security
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True
SECURE_BROWSER_XSS_FILTER = True


#Models.py
from django.db import models

class Question(models.Model):
    text = models.CharField(max_length=255)

    def __str__(self):
        return self.text

class Choice(models.Model):
```

```
    question = models.ForeignKey(Question, on_delete=models.CASCADE,
related_name="choices")
    text = models.CharField(max_length=255)
    is_correct = models.BooleanField(default=False)

    def __str__(self):
        return self.text
```

Migrate database:
python manage.py makemigrations
python manage.py migrate

#admin.py

```
from django.contrib import admin
from .models import Question, Choice

admin.site.register(Question)
admin.site.register(Choice)
```

#views.py

```
from django.shortcuts import render
from .models import Question, Choice

def quiz_view(request):
    questions = Question.objects.all()
    return render(request, "quiz/quiz.html", {"questions": questions})

def result_view(request):
    if request.method == "POST":
        score = 0
        questions = Question.objects.all()
        for question in questions:
            # Each question's input name should be its ID in the template
            selected = request.POST.get(str(question.id))
            if selected:
                try:
                    choice = Choice.objects.get(id=int(selected), question=question)
                    if choice.is_correct:
                        score += 1
                except Choice.DoesNotExist:
                    pass
```

```
        return render(request, "quiz/result.html", {
            "score": score,
            "total": questions.count()
        })
    else:
        # If someone visits result_view directly without POST
        return render(request, "quiz/result.html", {
            "score": 0,
            "total": 0
        })
```

```
#quiz/urls.py(app level)
from django.urls import path
from . import views

urlpatterns = [
    path('', views.quiz_view, name='quiz'),
    path('result/', views.result_view, name='result'),
]
```

```
#quiz_app/urls.py( Project level)
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('quiz.urls')),
]
```

Templates- quiz/templates/quiz/quiz.html

```
<form method="post" action="{% url 'result' %}">
    {% csrf_token %}
    {% for question in questions %}
        <h3>{{ question.text }}</h3>
        {% for choice in question.choices.all %}
            <input type="radio" name="{{ question.id }}" value="{{ choice.id }}">
            {{ choice.text }}<br>
        {% endfor %}
    {% endfor %}
    <button type="submit">Submit</button>
</form>
```

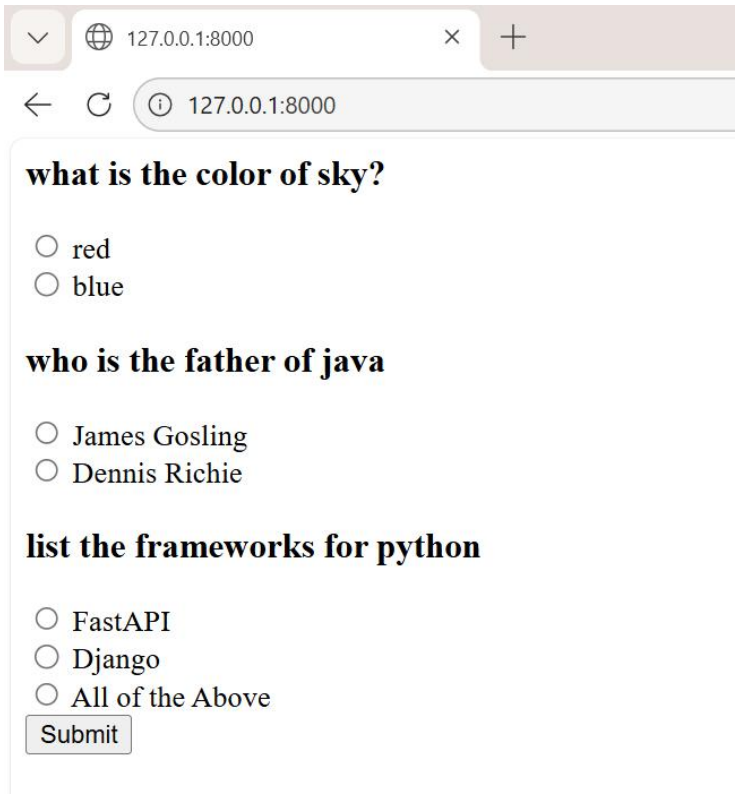
Results.html

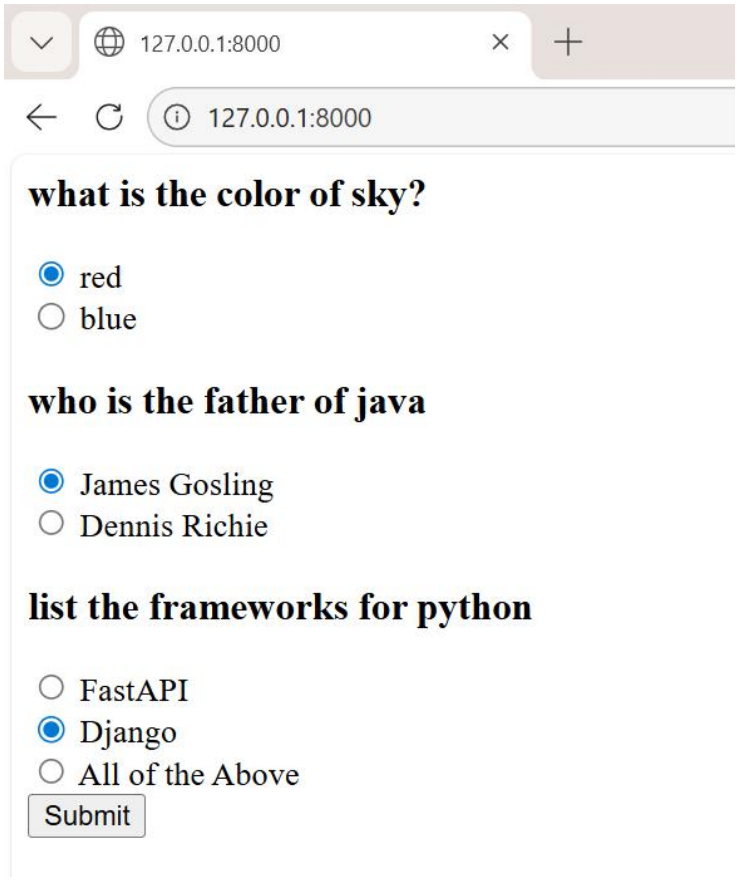
```
<h2>Your Score: {{ score }} / {{ total }}</h2>
```

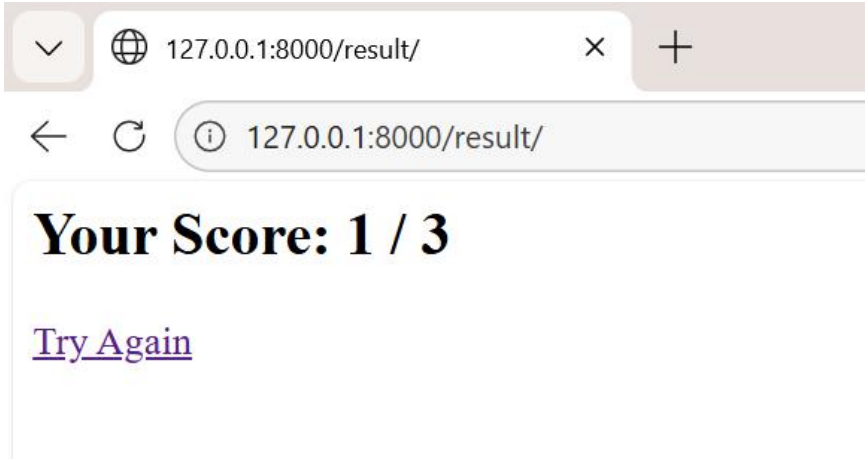
Try Again

Run Project:
python manage.py runserver

OUTPUT:




127.0.0.1:8000
127.0.0.1:8000
what is the color of sky?
red
blue
who is the father of java
James Gosling
Dennis Richie
list the frameworks for python
FastAPI
Django
All of the Above
Submit


127.0.0.1:8000/result/
127.0.0.1:8000/result/
Your Score: 1 / 3
Try Again

WEEK8: Create a Django application using generic views and forms

Generic views are part of Django's class-based views (CBVs) system. They abstract common patterns like listing objects, showing details, creating, updating, and deleting records.

View Type	Purpose	Class Name
List View	Display a list of objects	`ListView`
Detail View	Show details of a single object	`DetailView`
Create View	Handle form for creating an object	`CreateView`
Update View	Handle form for updating an object	`UpdateView`
Delete View	Confirm and delete an object	`DeleteView`

Generic forms refer to Django's built-in form classes that simplify form creation and handling.

Form Type	Description
`Form`	Basic form with manually defined fields
`ModelForm`	Auto-generates fields from a Django model
`FormView`	Generic view for displaying and processing forms

PROGRAM:

Step1: create a project and app

#Create an Environment

python –m venv week8

cd week8/scripts/activate

```
django-admin startproject library
cd library
python manage.py startapp bookmanager
```

Add 'bookmanager' to INSTALLED_APPS in library/settings.py.

Step2: Models (bookmanager/models.py)

```
from django.db import models
class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.CharField(max_length=100)
    published_date = models.DateField()
    genre = models.CharField(max_length=50)
    def __str__(self):
        return self.title
```

step3: Run These following commands

```
python manage.py makemigrations
python manage.py migrate
```

step4: Forms (bookmanager/forms.py)

```
from django import forms
from .models import Book
class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['title', 'author', 'published_date', 'genre']
```

Step5: Views (bookmanager/Views.py)

```
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView

from django.urls import reverse_lazy

from .models import Book

from .forms import BookForm

class BookListView(ListView):

    model = Book

    template_name = 'book_list.html'

class BookDetailView(DetailView):

    model = Book

    template_name = 'book_detail.html'

class BookCreateView(CreateView):

    model = Book

    form_class = BookForm

    template_name = 'book_form.html'

    success_url = reverse_lazy('book_list')

class BookUpdateView(UpdateView):

    model = Book

    form_class = BookForm

    template_name = 'book_form.html'

    success_url = reverse_lazy('book_list')

class BookDeleteView(DeleteView):
```

```
    model = Book
    template_name = 'book_confirm_delete.html'
    success_url = reverse_lazy('book_list')
```

Step6: URLs (bookmanager/urls.py)

```
from django.urls import path
from .views import BookListView, BookDetailView, BookCreateView, BookUpdateView, BookDeleteView
urlpatterns = [
    path('', BookListView.as_view(), name='book_list'),
    path('book/<int:pk>/', BookDetailView.as_view(), name='book_detail'),
    path('book/add/', BookCreateView.as_view(), name='book_add'),
    path('book/<int:pk>/edit/', BookUpdateView.as_view(), name='book_edit'),
    path('book/<int:pk>/delete/', BookDeleteView.as_view(), name='book_delete'),
]
```

Step7: Include this in the main library/urls.py:

```
from django.contrib import admin
from django.urls import path, include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('bookmanager.urls')),
]
```

Step8: in bookmanager/templates folder

8. 1.Base.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Book Manager</title>
</head>
<body>
    <h1>Book Manager</h1>
    <nav>
        <a href="{% url 'book_list' %}">Home</a> |
        <a href="{% url 'book_add' %}">Add Book</a>
    </nav>
    <hr>
    {% block content %}{% endblock %}
</body>
</html>
```

8.2 Book_list.html

```
{% extends 'base.html' %}
{% block content %}
<h2>Book List</h2>
<ul>
    {% for book in object_list %}
        <li>
            <a href="{% url 'book_detail' book.pk %}">{{ book.title }}</a> by
{{ book.author }}
```

```
        </li>
    {% empty %}
        <li>No books available.</li>
    {% endfor %}
</ul>
{% endblock %}
```

8.3 :book_detail.html

```
{% extends 'base.html' %}
{% block content %}
<h2>{{ object.title }}</h2>
<p><strong>Author:</strong> {{ object.author }}</p>
<p><strong>Published:</strong> {{ object.published_date }}</p>
<p><strong>Genre:</strong> {{ object.genre }}</p>
<a href="{% url 'book_edit' object.pk %}">Edit</a> |
<a href="{% url 'book_delete' object.pk %}">Delete</a>
{% endblock %}
```

8.4 book_form.html

```
{% extends 'base.html' %}
{% block content %}
<h2>{% if object %}Edit{% else %}Add{% endif %} Book</h2>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Save</button>
```

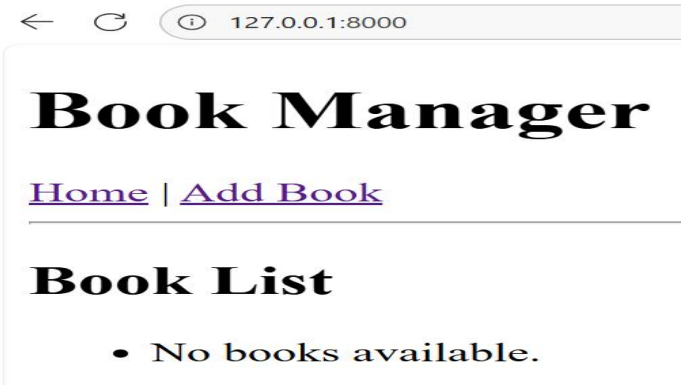
```
</form>
{% endblock %}
```

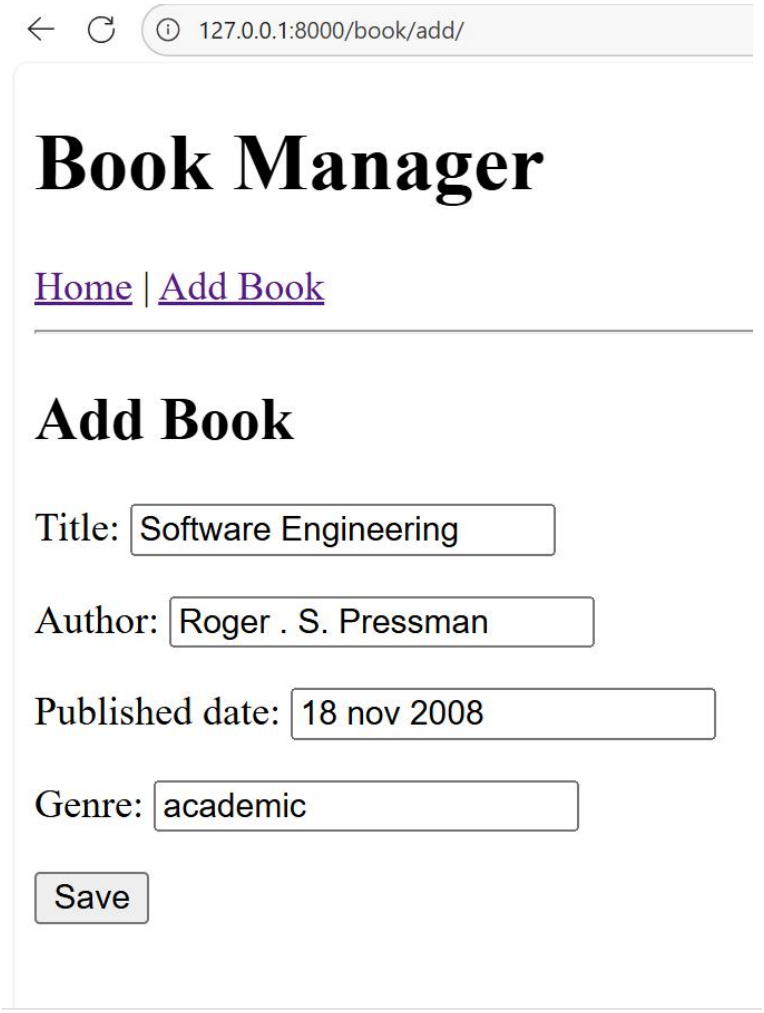
8.5 book_confirm-delete.html

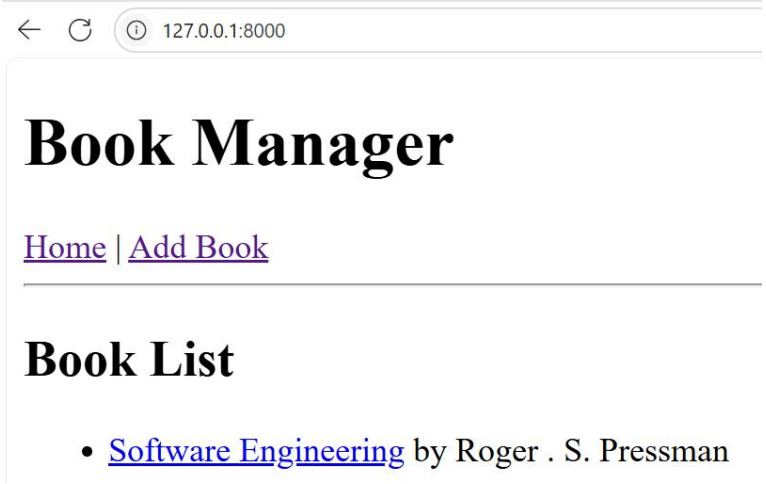
```
{% extends 'base.html' %}
{% block content %}
<h2>Confirm Delete</h2>
<p>Are you sure you want to delete "{{ object.title }}"?</p>
<form method="post">
    {% csrf_token %}
    <button type="submit">Yes, delete</button>
    <a href="{% url 'book_list' %}">Cancel</a>
</form>
{% endblock %}
```

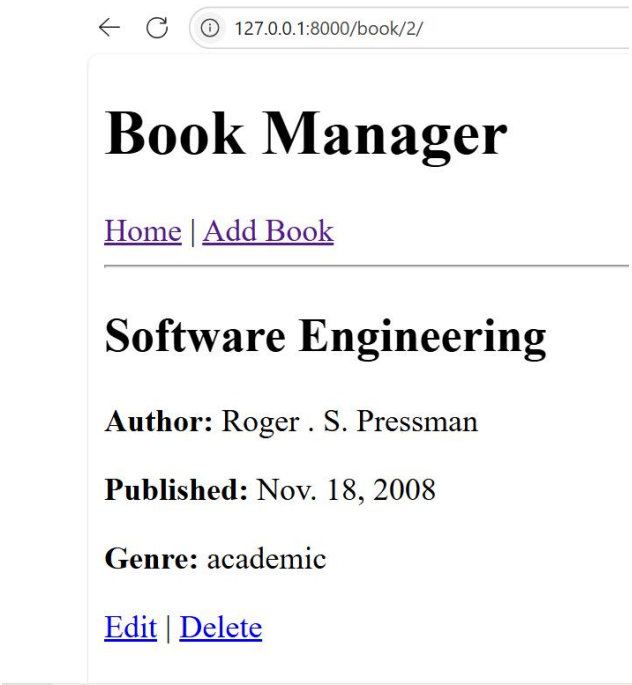
Run application: python manage.py runserver

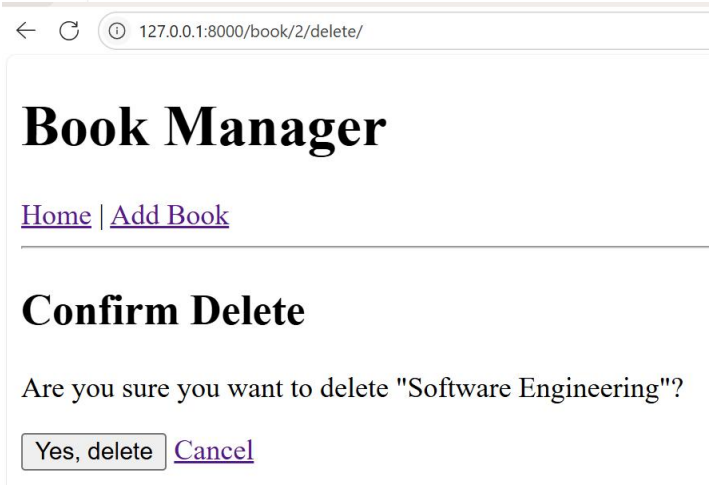
OUTPUT:




127.0.0.1:8000/book/add/
Book Manager
Home | Add Book
Add Book
Title: Software Engineering
Author: Roger . S. Pressman
Published date: 18 nov 2008
Genre: academic
Save


127.0.0.1:8000
Book Manager
Home | Add Book
Book List
Software Engineering by Roger . S. Pressman


127.0.0.1:8000/book/2/
Book Manager
Home | Add Book
Software Engineering
Author: Roger . S. Pressman
Published: Nov. 18, 2008
Genre: academic
Edit | Delete


127.0.0.1:8000/book/2/delete/
Book Manager
Home | Add Book
Confirm Delete
Are you sure you want to delete "Software Engineering"?
Yes, delete
Cancel

WEEK9: Create an Application for Working with Mails using Django Framework

Steps:

#Create an Environment

python –m venv week9

cd week9/scripts/activate

Install Django

pip install django

Create project

django-admin startproject mail_app

cd mail_app

Create mail application

python manage.py startapp mail

```
mail_app/                    # Root project folder
│
├── manage.py                 # Django command-line utility
│
├── mail_app/                 # Project settings & configuration
│   ├── __init__.py
│   ├── settings.py           # Global settings (DB, email backend, auth)
│   ├── urls.py               # Root URL routing
│   ├── asgi.py               # ASGI entry point
│   └── wsgi.py               # WSGI entry point
│
├── mail/                     # Our mail application
│   ├── __init__.py
│   ├── admin.py              # Register models for Django admin
```

```
│   ├── apps.py               # App configuration
│   ├── models.py             # Mail model (sender, recipient, subject, body)
│   ├── forms.py              # Forms (MailForm, RegisterForm)
│   ├── views.py              # Business logic (inbox, compose, login, register)
│   ├── urls.py               # App-specific URL routing
│   ├── tests.py              # Unit tests
│   ├── migrations/           # Auto-generated DB migrations
│   │   └── __init__.py
│   └── templates/mail/       # HTML templates
│       ├── inbox.html        # Inbox page
│       ├── compose.html      # Compose mail page
│       ├── login.html        # Login page
│       └── register.html     # Register page
│
├── db.sqlite3                # SQLite3 database file
└── static/                   # (Optional) static files (CSS, JS, images)
    └── mail/                 # App-specific static files
        ├── style.css         # Styling for pages
        └── scripts.js        # Optional JS
```

INSTALLED_APPS = [

```
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
```

```
    'django.contrib.staticfiles',
    'mail',   # our app
]

# Email backend (SMTP example: Gmail)
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = 'your_email@gmail.com'
EMAIL_HOST_PASSWORD = 'your_password_or_app_password'

LOGIN_REDIRECT_URL = '/inbox/'   # after login, go to inbox
LOGOUT_REDIRECT_URL = '/login/' # after logout, go to login page

#models.py
from django.db import models
from django.contrib.auth.models import User
class Mail(models.Model):
    sender = models.EmailField()
    recipient = models.EmailField()
    subject = models.CharField(max_length=255)
    body = models.TextField()
    timestamp = models.DateTimeField(auto_now_add=True)
    owner = models.ForeignKey(User, on_delete=models.CASCADE)
    def __str__(self):
```

```
        return f"{self.subject} ({self.sender} → {self.recipient})"
#forms.py
from django import forms
from django.contrib.auth.models import User
from django.contrib.auth.forms import UserCreationForm

class RegisterForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ["username", "email", "password1", "password2"]



#views.py
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
from django.contrib.auth import login, logout
from django.contrib.auth.forms import AuthenticationForm
from django.core.mail import send_mail
from django.conf import settings

from .forms import MailForm, RegisterForm
from .models import Mail
```

```
#    Inbox View
@login_required
def inbox_view(request):
    mails = Mail.objects.filter(owner=request.user).order_by('-timestamp')
    return render(request, "mail/inbox.html", {"mails": mails})


# ✉ Compose Mail View
@login_required
def compose_view(request):
    if request.method == "POST":
        form = MailForm(request.POST)
        if form.is_valid():
            recipient = form.cleaned_data['recipient']
            subject = form.cleaned_data['subject']
            body = form.cleaned_data['body']

            # Send email via SMTP
            send_mail(subject, body, settings.EMAIL_HOST_USER, [recipient])

            # Save to DB
            Mail.objects.create(
                sender=settings.EMAIL_HOST_USER,
                recipient=recipient,
                subject=subject,
```

```
            body=body,
            owner=request.user
        )
        return redirect("inbox")
    else:
        form = MailForm()
    return render(request, "mail/compose.html", {"form": form})


#    Login View
def login_view(request):
    if request.method == "POST":
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            return redirect("inbox")
    else:
        form = AuthenticationForm()
    return render(request, "mail/login.html", {"form": form})


#    Logout View
@login_required
def logout_view(request):
    logout(request)
```

```
    return redirect("login")



#    Register View
def register_view(request):
    if request.method == "POST":
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)  # auto-login after registration
            return redirect("inbox")
    else:
        form = RegisterForm()
    return render(request, "mail/register.html", {"form": form})

urls.py (applevel)
from django.urls import path
from . import views

urlpatterns = [
    path('inbox/', views.inbox_view, name='inbox'),
    path('compose/', views.compose_view, name='compose'),
    path('login/', views.login_view, name='login'),   # already added earlier
    path('logout/', views.logout_view, name='logout'),
    path('register/', views.register_view, name='register'),  # new
]
```

Urls.py(project level)

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('mail.urls')),
]
```

Templates

Register.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Register</title>
    <link rel="stylesheet" href="{% static 'mail/style.css' %}">
    <script src="{% static 'mail/script.js' %}"></script>
</head>
<body>
    <h2>Create an Account</h2>

    <!-- Registration Form -->
    <form method="post">
```

```
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Register</button>
    </form>

    <!-- Navigation -->
    <p>Already have an account? <a href="{% url 'login' %}">Login here</a></p>
</body>
</html>
```

Login.html

```
<p>Don't have an account? <a href="{% url 'register' %}">Register here</a></p>
```

Inbox.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Inbox</title>
    <link rel="stylesheet" href="{% static 'mail/style.css' %}">
    <script src="{% static 'mail/script.js' %}"></script>
</head>
<body>
    <h2>Your Inbox</h2>

    <!-- Navigation -->
```

```
    <p>
        <a href="{% url 'compose' %}">Compose Mail</a> |
        <a href="{% url 'logout' %}">Logout</a>
    </p>

    <!-- Inbox List -->
    <ul>
        {% for mail in mails %}
            <li>
                <b>{{ mail.subject }}</b><br>
                From: {{ mail.sender }} → To: {{ mail.recipient }}<br>
                <small>{{ mail.timestamp }}</small><br>
                <p>{{ mail.body|truncatewords:20 }}</p>
            </li>
        {% empty %}
            <li>No mails yet.</li>
        {% endfor %}
    </ul>
</body>
</html>
```

Compose .html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Compose Mail</title>
    <link rel="stylesheet" href="{% static 'mail/style.css' %}">
    <script src="{% static 'mail/script.js' %}"></script>
</head>
<body>
    <h2>Compose Mail</h2>

    <!-- Navigation -->
    <p>
        <a href="{% url 'inbox' %}">Inbox</a> |
        <a href="{% url 'logout' %}">Logout</a>
    </p>

    <!-- Compose Form -->
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Send Mail</button>
    </form>
</body>
</html>
```

Login.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <title>Login</title>
    <link rel="stylesheet" href="{% static 'mail/style.css' %}">
    <script src="{% static 'mail/script.js' %}"></script>
</head>
<body>
    <h2>Login</h2>

    <!-- Login Form -->
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Login</button>
    </form>

    <!-- Navigation -->
    <p>Don't have an account? <a href="{% url 'register' %}">Register here</a></p>
</body>
</html>
```

Logout.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Logout</title>
    <link rel="stylesheet" href="{% static 'mail/style.css' %}">
</head>
<body>
    <h2>You have been logged out</h2>
    <p><a href="{% url 'login' %}">Login again</a></p>
</body>
</html>
```

Styles.css

```
/* General page styling */
body {
    font-family: Arial, sans-serif;
    background-color: #f4f6f9;
    margin: 0;
    padding: 0;
}

/* Header */
h2 {
    color: #333;
    text-align: center;
    margin-top: 20px;
}
```

```
/* Form styling */
form {
    max-width: 400px;
    margin: 20px auto;
    padding: 20px;
    background: #fff;
    border-radius: 8px;
    box-shadow: 0px 2px 6px rgba(0,0,0,0.1);
}

form input, form textarea, form button {
    width: 100%;
    padding: 10px;
    margin: 8px 0;
    border-radius: 5px;
    border: 1px solid #ccc;
}

/* Buttons */
button {
    background-color: #007bff;
    color: white;
    border: none;
    cursor: pointer;
    font-weight: bold;
}
```

```
button:hover {
    background-color: #0056b3;
}

/* Inbox list */
ul {
    list-style-type: none;
    padding: 0;
}

ul li {
    background: #fff;
    margin: 10px auto;
    padding: 15px;
    border-radius: 6px;
    max-width: 600px;
    box-shadow: 0px 2px 4px rgba(0,0,0,0.1);
}

ul li b {
    color: #007bff;
}

/* Navigation links */
a {
```

```
    color: #007bff;
    text-decoration: none;
}

a:hover {
    text-decoration: underline;
}
```

Script.js

```
// Wait until the DOM is fully loaded
document.addEventListener("DOMContentLoaded", function() {

    // Confirmation before sending mail
    const composeForm = document.querySelector("form");
    if (composeForm && window.location.pathname.includes("compose")) {
        composeForm.addEventListener("submit", function(event) {
            const confirmSend = confirm("Are you sure you want to send this mail?");
            if (!confirmSend) {
                event.preventDefault(); // cancel submission
            } else {
                // Show toast notification after sending
                showToast("✅ Mail sent successfully!");
            }
        });
    }

    //    Highlight inbox items on hover
```

```
const inboxItems = document.querySelectorAll("ul li");
inboxItems.forEach(item => {
    item.addEventListener("mouseover", () => {
        item.style.backgroundColor = "#e9f5ff"; // light blue highlight
    });
    item.addEventListener("mouseout", () => {
        item.style.backgroundColor = "#fff"; // reset
    });
});

// Auto-focus first input field (login/register/compose)
const firstInput = document.querySelector("form input");
if (firstInput) {
    firstInput.focus();
}

// Password visibility toggle (login/register forms)
const passwordFields = document.querySelectorAll("input[type='password']");
passwordFields.forEach(field => {
    const toggle = document.createElement("span");
    toggle.textContent = "   ";
    toggle.style.cursor = "pointer";
    toggle.style.marginLeft = "10px";
    field.parentNode.appendChild(toggle);

    toggle.addEventListener("click", () => {
```

```
            field.type = field.type === "password" ? "text" : "password";
        });
    });

    //    Toast notification function
    function showToast(message) {
        const toast = document.createElement("div");
        toast.textContent = message;
        toast.style.position = "fixed";
        toast.style.bottom = "20px";
        toast.style.right = "20px";
        toast.style.background = "#28a745";
        toast.style.color = "#fff";
        toast.style.padding = "10px 20px";
        toast.style.borderRadius = "5px";
        toast.style.boxShadow = "0px 2px 6px rgba(0,0,0,0.2)";
        document.body.appendChild(toast);

        setTimeout(() => toast.remove(), 3000); // disappear after 3s
    }
});
```

Run application

python manage.py createsuperuser

python manage.py collectstatic

python manage.py runserver


127.0.0.1:8000/compose/
Compose Mail
Recipient:
Subject:
Body:
Send

WEEK10: Create an application that fetches the weather information from openweathermap.org and the display the current and historical weather information using graphical representation using chart.js implement using Django

Project Strucuture:

```
weather_project/
├─ manage.py
├─ .env
├─ requirements.txt
├─ weather_project/
│  ├─ __init__.py
│  ├─ settings.py
│  ├─ urls.py
│  ├─ asgi.py
│  └─ wsgi.py
└─ weather/
   ├─ __init__.py
   ├─ apps.py
   ├─ urls.py
   ├─ views.py
   ├─ services.py
```

```
├── models.py
├── forms.py
├── templates/
│   └── weather/
│       └── dashboard.html
└── static/
    └── weather/
        ├── styles.css
        └── charts.js
```

Step1:

Create environment

pip install django

pip install django requests

then create django project with weather_Project

Create weather app

django-admin startproject weather_project

cd weather_project

python manage.py startapp weather_app

Step2: create a text file

Requirements.txt

Django>=4.2[new version]

requests>=2.31

python-dotenv>=1.0

in .env file

DJANGO_SECRET_KEY=your-dev-secret

OPENWEATHER_API_KEY=your_openweather_api_key

DEBUG=True

ALLOWED_HOSTS=localhost,127.0.0.1

Settings.py

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "weather",
]
```

Project url-weather_project/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("weather.urls")),
]
```

Applevel- weather/urls.py

```
from django.urls import path
from .views import dashboard
urlpatterns = [
    path("", dashboard, name="dashboard"),
]
```

Step3:

Utils.py

```
# utils.py
import requests
from django.conf import settings
```

```
def fetch_weather(city, country_code="IN"):
    api_key = settings.OPENWEATHER_API_KEY
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city},{country_code}&appid={api_key}&units=metric"
    response = requests.get(url)
    data = response.json()
    return {
        "city": city,
        "country_code": country_code,
        "temp_c": data["main"]["temp"],
        "humidity": data["main"]["humidity"],
        "wind_speed": data["wind"]["speed"],
    }
```

Forms.py

```
from django import forms
from .models import WeatherSnapshot


class CityForm(forms.Form):
    city = forms.ChoiceField(label="Choose City")
```

```
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Populate dropdown with distinct cities from WeatherSnapshot
        cities = WeatherSnapshot.objects.values_list("city", flat=True).distinct()
        self.fields["city"].choices = [(c, c) for c in cities]
```

Models.py

```
from django.db import models
from django.core.validators import MinValueValidator, MaxValueValidator


class WeatherSnapshot(models.Model):
    city = models.CharField(max_length=100)
    country_code = models.CharField(max_length=2, blank=True)  # e.g., "IN"
    timestamp = models.DateTimeField(auto_now_add=True)

    temp_c = models.FloatField(
        validators=[MinValueValidator(-100), MaxValueValidator(60)]
    )
    humidity = models.IntegerField(
        validators=[MinValueValidator(0), MaxValueValidator(100)]
```

```
    )
    wind_speed = models.FloatField(
        validators=[MinValueValidator(0)]
    )

    class Meta:
        ordering = ["-timestamp"]
        indexes = [
            models.Index(fields=["city"]),
            models.Index(fields=["timestamp"]),
        ]

    def __str__(self):
        return f"{self.city} ({self.country_code}) - {self.temp_c}°C at {self.timestamp:%Y-%m-%d %H:%M}"
```

services.py

```
import requests
from django.conf import settings

BASE_URL = "https://api.openweathermap.org/data/2.5"
```

```
API_KEY = settings.OPENWEATHER_API_KEY


def get_current_weather(city: str, country_code: str = ""):
    if not API_KEY:
        raise RuntimeError("Missing OPENWEATHER_API_KEY")
    q = f"{city},{country_code}" if country_code else city
    url = f"{BASE_URL}/weather"
    params = {"q": q, "appid": API_KEY, "units": "metric"}
    r = requests.get(url, params=params, timeout=10)
    r.raise_for_status()
    data = r.json()
    return {
        "city": data["name"],
        "lat": data["coord"]["lat"],
        "lon": data["coord"]["lon"],
        "temp_c": data["main"]["temp"],
        "humidity": data["main"]["humidity"],
        "wind_speed": data["wind"]["speed"],
        "description": data["weather"][0]["description"].title(),
        "icon": data["weather"][0]["icon"],
```

```
    }


def get_forecast_5day(city: str, country_code: str = ""):
    if not API_KEY:
        raise RuntimeError("Missing OPENWEATHER_API_KEY")
    q = f"{city},{country_code}" if country_code else city
    url = f"{BASE_URL}/forecast"
    params = {"q": q, "appid": API_KEY, "units": "metric"}
    r = requests.get(url, params=params, timeout=10)
    r.raise_for_status()
    data = r.json()
    points = []
    for item in data.get("list", []):
        points.append({
            "dt_txt": item["dt_txt"],
            "temp_c": item["main"]["temp"],
            "humidity": item["main"]["humidity"],
            "wind_speed": item["wind"]["speed"],
        })
    return points
```

views.py

```
# views.py
from django.shortcuts import render
from django.utils.timezone import localtime
from .models import WeatherSnapshot
from .forms import CityForm

def weather_chart_view(request):
    city = None
    labels, temp_data, humidity_data, wind_data = [], [], [], []

    if request.method == "POST":
        form = CityForm(request.POST)
        if form.is_valid():
            city = form.cleaned_data["city"]
            qs = WeatherSnapshot.objects.filter(city=city).order_by("-timestamp")[:10]
            qs = list(reversed(qs))

            labels = [localtime(s.timestamp).strftime("%Y-%m-%d %H:%M") for s in
qs]
```

```
            temp_data = [s.temp_c for s in qs]
            humidity_data = [s.humidity for s in qs]
            wind_data = [s.wind_speed for s in qs]
    else:
        form = CityForm()

    return render(request, "weather/chart.html", {
        "form": form,
        "city": city,
        "labels": labels,
        "temp_data": temp_data,
        "humidity_data": humidity_data,
        "wind_data": wind_data,
    })
```

step4:

templates folder -dashboard.html

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="utf-8" />
  <title>Weather dashboard</title>
  <link rel="stylesheet" href="{% static 'weather/styles.css' %}">
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
  <main class="container">
    <h1>Weather dashboard</h1>

    <form method="post" class="city-form">
      {% csrf_token %}
      {{ form.as_p }}
      <button type="submit">Update</button>
    </form>

    {% if messages %}
      <ul class="messages">
        {% for m in messages %}
          <li class="{{ m.tags }}">{{ m }}</li>
```

```
      {% endfor %}
    </ul>
  {% endif %}
  {% if current %}
    <section class="current">
      <h2>Current weather — {{ current.city }}</h2>
      <div class="current-grid">
        <div>
          <p><strong>Temperature:</strong> {{ current.temp_c }} °C</p>
          <p><strong>Humidity:</strong> {{ current.humidity }} %</p>
          <p><strong>Wind:</strong> {{ current.wind_speed }} m/s</p>
          <p><strong>Conditions:</strong> {{ current.description }}</p>
        </div>
        <div>
          <img                                                    alt="icon"
src="https://openweathermap.org/img/wn/{{ current.icon }}@2x.png">
        </div>
      </div>
    </section>
  {% endif %}
```

```
    <section class="charts">
      <h2>5-day forecast (3-hour intervals)</h2>
      <canvas id="forecastTempChart"></canvas>
      <canvas id="forecastHumidityChart"></canvas>
      <h2>Local historical snapshots (last 30)</h2>
      <canvas id="historicalTempChart"></canvas>
      <canvas id="historicalWindChart"></canvas>
    </section>
  </main>

  {{ forecast_points|json_script:"forecast-data" }}
  {{ historical_points|json_script:"historical-data" }}
  <script src="{% static 'weather/charts.js' %}"></script>
</body>
</html>
```

Step5: Static/charts.js

```
<h2>Weather Trends</h2>

<canvas id="weatherChart"></canvas>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

```
<script>
  // These arrays come from your Django context
  const labels = ["2026-01-20 18:00", "2026-01-20 19:00", "2026-01-20 20:00"];
  const tempData = [24.5, 23.8, 22.9];
  const humidityData = [60, 65, 70];
  const windData = [3.2, 2.8, 2.5];

  new Chart(document.getElementById('weatherChart'), {
    type: 'line',
    data: {
      labels: labels,
      datasets: [
        {
          label: 'Temperature (°C)',
          data: tempData,
          borderColor: 'red',
          backgroundColor: 'rgba(255,0,0,0.2)',
          fill: false,
          tension: 0.3
        },
```

```
      {
        label: 'Humidity (%)',
        data: humidityData,
        borderColor: 'blue',
        backgroundColor: 'rgba(0,0,255,0.2)',
        fill: false,
        tension: 0.3
      },
      {
        label: 'Wind Speed (m/s)',
        data: windData,
        borderColor: 'green',
        backgroundColor: 'rgba(0,255,0,0.2)',
        fill: false,
        tension: 0.3
      }
    ]
  },
  options: {
    responsive: true,
```

```
    plugins: {
      legend: { position: 'top' },
      tooltip: { enabled: true }
    },
    scales: {
      x: {
        title: { display: true, text: 'Timestamp' }
      },
      y: {
        title: { display: true, text: 'Values' }
      }
    }
  }
});
</script>
```

Styles.css

```
body {
font-family: system-ui, sans-serif; margin: 0; background: #f7f9fb; color: #2c3e50;
}
```

.container { max-width: 1000px; margin: 2rem auto; padding: 0 1rem; }

.city-form { display: flex; gap: 1rem; align-items: flex-end; }

.city-form p { margin: 0; }

.current-grid { display: grid; grid-template-columns: 1fr 120px; gap: 1rem; align-items: center; }

.messages { list-style: none; padding: 0; }

.messages li { margin: .5rem 0; padding: .5rem .75rem; border-radius: 6px; }

.messages li.error { background: #fdecea; color: #c0392b; }

canvas { margin: 1rem 0; background: #fff; border: 1px solid #eaecef; border-radius: 8px; padding: .5rem; }

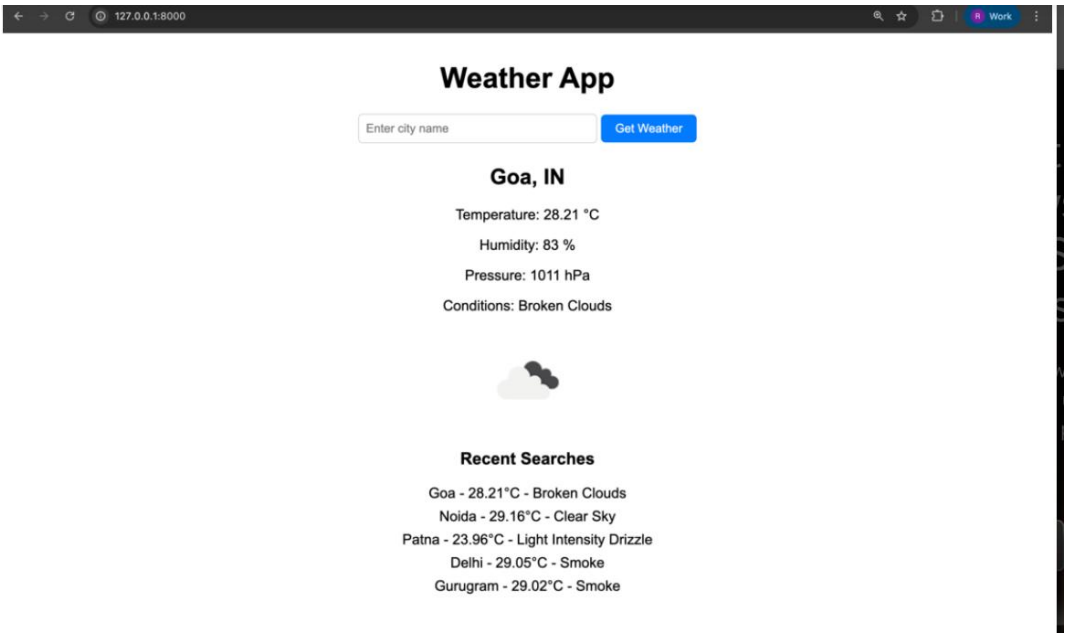
Run the application

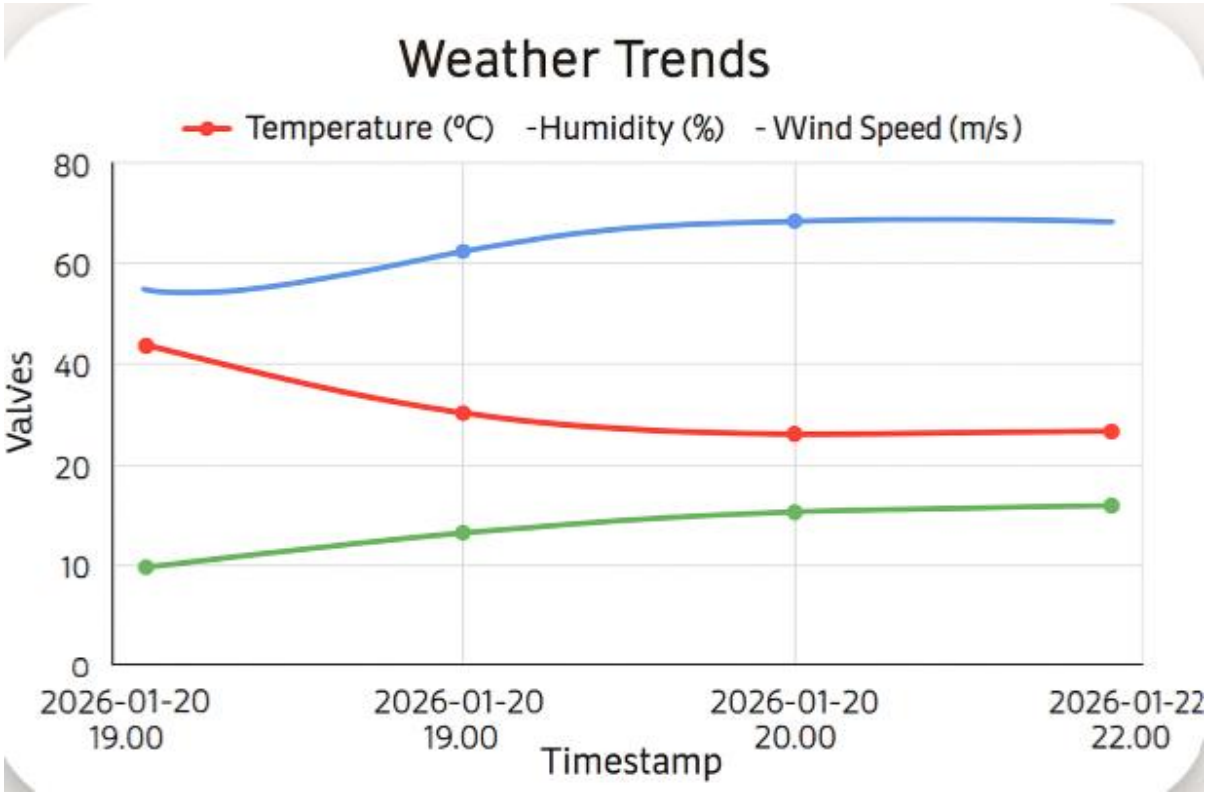
Install the dependencies

pip install -r requirements.txt

- Add your OpenWeatherMap API key to .env.
- Make migrations and migrate

python manage.py runserver


127.0.0.1:8000
Work
Weather App
Enter city name
Get Weather
Goa, IN
Temperature: 28.21 °C
Humidity: 83 %
Pressure: 1011 hPa
Conditions: Broken Clouds
Recent Searches
Goa - 28.21°C - Broken Clouds
Noida - 29.16°C - Clear Sky
Patna - 23.96°C - Light Intensity Drizzle
Delhi - 29.05°C - Smoke
Gurugram - 29.02°C - Smoke


Weather Trends
Temperature (°C)
Humidity (%)
Wind Speed (m/s)
Values
80
60
40
20
10
0
2026-01-20 19.00
2026-01-20 19.00
2026-01-20 20.00
2026-01-22 22.00
Timestamp

WEEK11: Case Study: Django Application for VJIT Student Portal and deploy it into GitHub.

Case study : VJIT STUDENT PORTAL

django-admin startproject vjit_site

cd vjit_site

python manage.py startapp main

Apps to create

main → Homepage, About, Contact
academics → Departments, Courses
admissions → Admission info, forms
events → News, Events, Announcements
students → Student portal, results, notices

Models.py

```
# academics/models.py

from django.db import models


class Department(models.Model):
    name = models.CharField(max_length=100)
    hod = models.CharField(max_length=100)
    description = models.TextField()
    contact_email = models.EmailField()


    def __str__(self):
        return self.name
```

views.py

```
# main/views.py
from django.shortcuts import render


def home(request):
    return render(request, 'main/home.html')
```

Project Level url mapping

```
# vjit_site/urls.py
from django.contrib import admin
from django.urls import path, include
from main import views as main_views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', main_views.home, name='home'),
    path('academics/', include('academics.urls')),
]
```

```
<!DOCTYPE html>
<html>
<head>
    <title>VJIT - Home</title>
</head>
<body>
    <h1>Welcome to Vidya Jyothi Institute of Technology</h1>
```

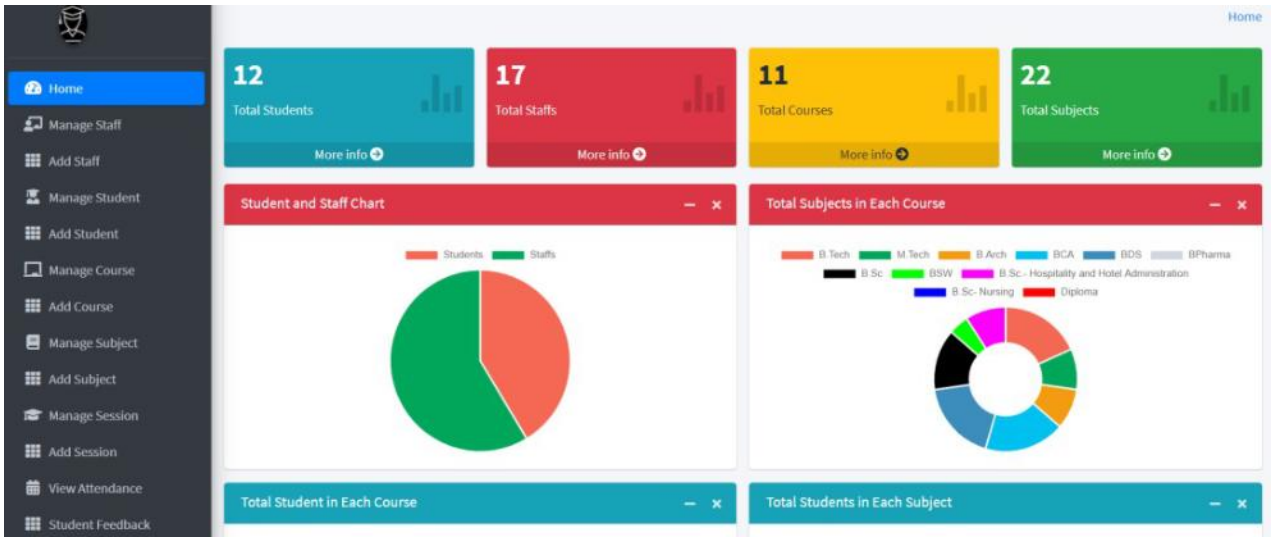
```
    <nav>
        <a href="/academics/">Academics</a> |
        <a href="/admissions/">Admissions</a> |
        <a href="/events/">Events</a>
    </nav>
</body>
</html>
```

Admin.py

```
# academics/admin.py
from django.contrib import admin
from .models import Department
admin.site.register(Department)
```

Run application

Python manage.py runserver



Step2: Deploying a Project to GitHub

1. Create a GitHub account
Install **Git** on your computer (git --version to check).

git init

git add .

git commit -m "Initial commit"

2. Create a repository on GitHub

Go to GitHub → click **New Repository**
Give it a name (e.g., VJIT-app).
Choose **Public** (so others can see it) or **Private**.

Link your local project to GitHub

git remote add origin https://github.com/your-username/vjit-app.git

git branch -M main

git push -u origin main

Hosting a Website with GitHub Pages

If your project is a **static site** (HTML, CSS, JS):

1. Push your code to GitHub as above.
2. Go to your repo → **Settings** → **Pages**.
3. Under "Source," select branch main and folder /root (or /docs).
4. Save → GitHub will give you a live URL like:

https://your-username.github.io/vjit-app/

GitHub itself **cannot run Django apps** (because Django needs Python + a server). GitHub Pages only hosts static HTML/CSS/JS.

- **PythonAnywhere** → Free tier, good for small Django apps **OR**
- **Heroku**: Free tier for Django apps, integrates with GitHub **OR**
- **Vercel/Render**: Modern hosting, supports Django with some setup

Deploy with Render

1. Push your Django project to GitHub (step 1 above).
2. Go to Render.
3. Create a new **Web Service** → connect your GitHub repo.
4. Set **Build Command**:

   pip install -r requirements.txt
   python manage.py migrate

   gunicorn vjit_project.wsgi:application

   Add environment variables

Deploy → Render gives you a live URL (e.g., https://vjit-django.onrender.com).